

Chapitre III : Conception

Objectif du chapitre : Ce chapitre presente la conception detaillee de la plateforme *Test Management Platform*. Il couvre l'ensemble des diagrammes UML necessaires a la comprehension de l'architecture, des interactions et du comportement du systeme. Chaque diagramme est accompagne d'une description textuelle detaillee.

Sommaire

1. Introduction

2. Diagramme de cas d'utilisation

- 2.1. Vue globale du systeme
- 2.2. Gerer le profil
- 2.3. Gerer les utilisateurs
- 2.4. Gerer les projets
- 2.5. Gerer les modules
- 2.6. Gerer les scenarios
- 2.7. Gerer les cas de test
- 2.8. Gerer les sessions de test
- 2.9. Gerer les executions
- 2.10. Gerer les anomalies
- 2.11. Dashboard et rapports
- 2.12. Fonctionnalites IA

3. Diagramme de classes

4. Diagrammes de sequence

- 4.1. Authentification
- 4.2. Processus d'execution de test
- 4.3. Generation de cas de test par IA
- 4.4. Resume intelligent de session

4.5. Suggestion de gravite par IA

5. Diagramme d'activite

6. Diagrammes d'etats-transitions

6.1. Etats d'un projet

6.2. Etats d'une anomalie

6.3. Etats d'une execution

7. Diagramme de composants

8. Diagramme de deploiement

9. Synthese

1. Introduction

La phase de conception constitue une etape fondamentale dans le cycle de developpement logiciel. Elle permet de traduire les besoins fonctionnels identifies lors de l'analyse en une architecture technique claire et detaillee. Dans ce chapitre, nous presentons la conception de notre plateforme de gestion de tests logiciels, en utilisant le langage de modelisation unifie UML (Unified Modeling Language).

Notre application repose sur une architecture trois-tiers composee d'un frontend React 18 avec Vite et Tailwind CSS, d'un backend Spring Boot 3.2.5 avec Java 17, et d'une base de donnees MySQL 8. Elle integre egalement des services externes tels que Cloudinary pour l'hebergement d'images, et Ollama (modele deepseek-r1:7b) pour les fonctionnalites d'intelligence artificielle.

Le systeme definit trois roles principaux avec des permissions distinctes :

Role	Responsabilites principales
Admin	Gestion des utilisateurs, creation et gestion des projets, assignation des membres, archivage, suppression des anomalies, dashboard global, gestion des archives
Manager	Validation des projets, gestion des modules/scenarios/cas de test, consultation des sessions et executions (lecture seule), changement de statut et commentaire sur les anomalies, dashboard global et par projet
Testeur	Gestion des modules/scenarios/cas de test, creation des sessions, execution des tests, declaration des anomalies, import Excel, generation IA, dashboard par projet

Les diagrammes presentes dans ce chapitre couvrent les perspectives statique, dynamique et physique du systeme, offrant ainsi une vision complete de l'architecture logicielle.

2. Diagrammes de cas d'utilisation

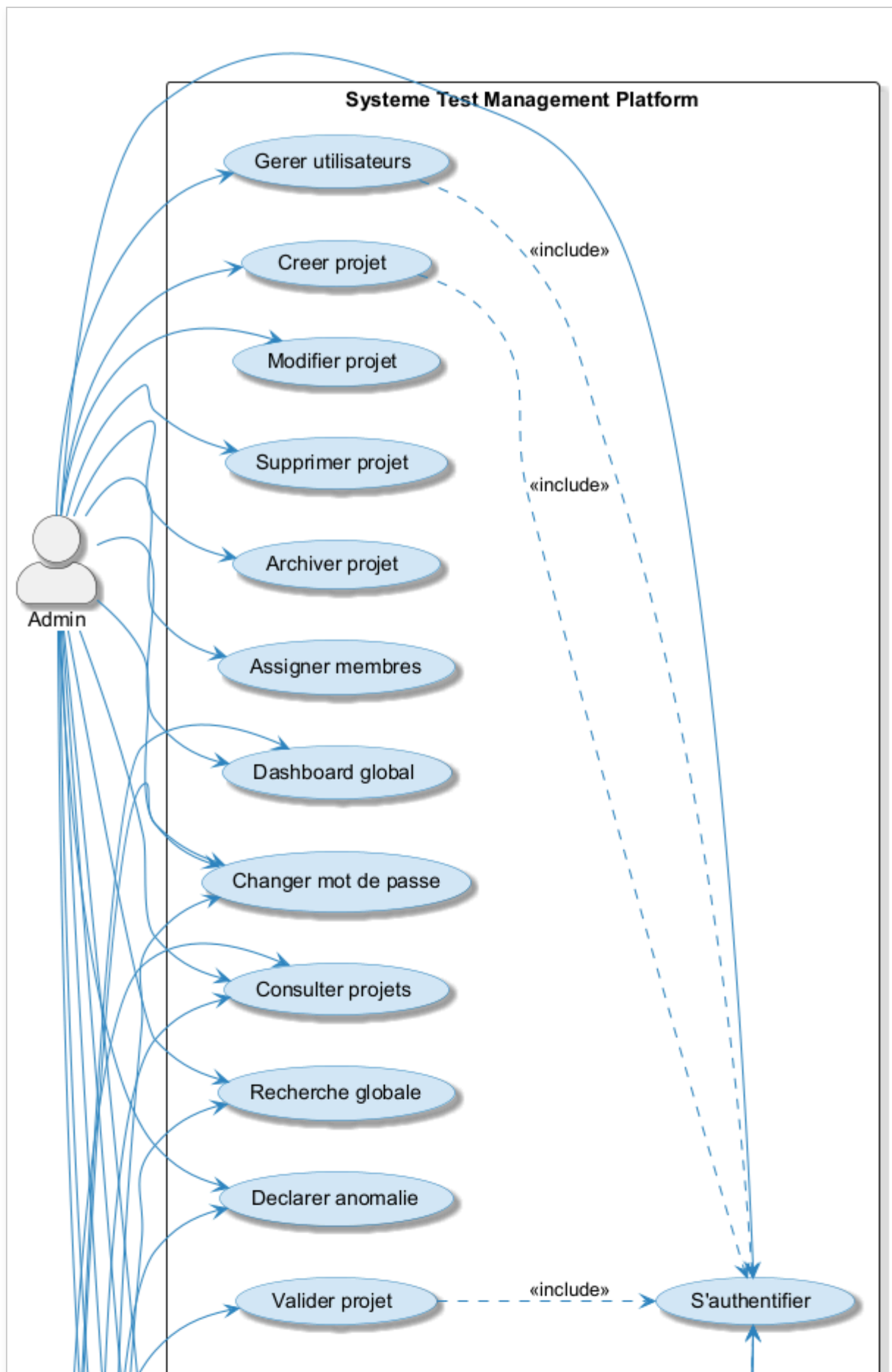
Les diagrammes de cas d'utilisation permettent de decire les fonctionnalites du systeme du point de vue des acteurs. Ils identifient les interactions entre les utilisateurs et l'application, en distinguant les droits d'accès de chaque role. Nous presentons d'abord une vue globale, puis un diagramme detaille pour chaque module fonctionnel.

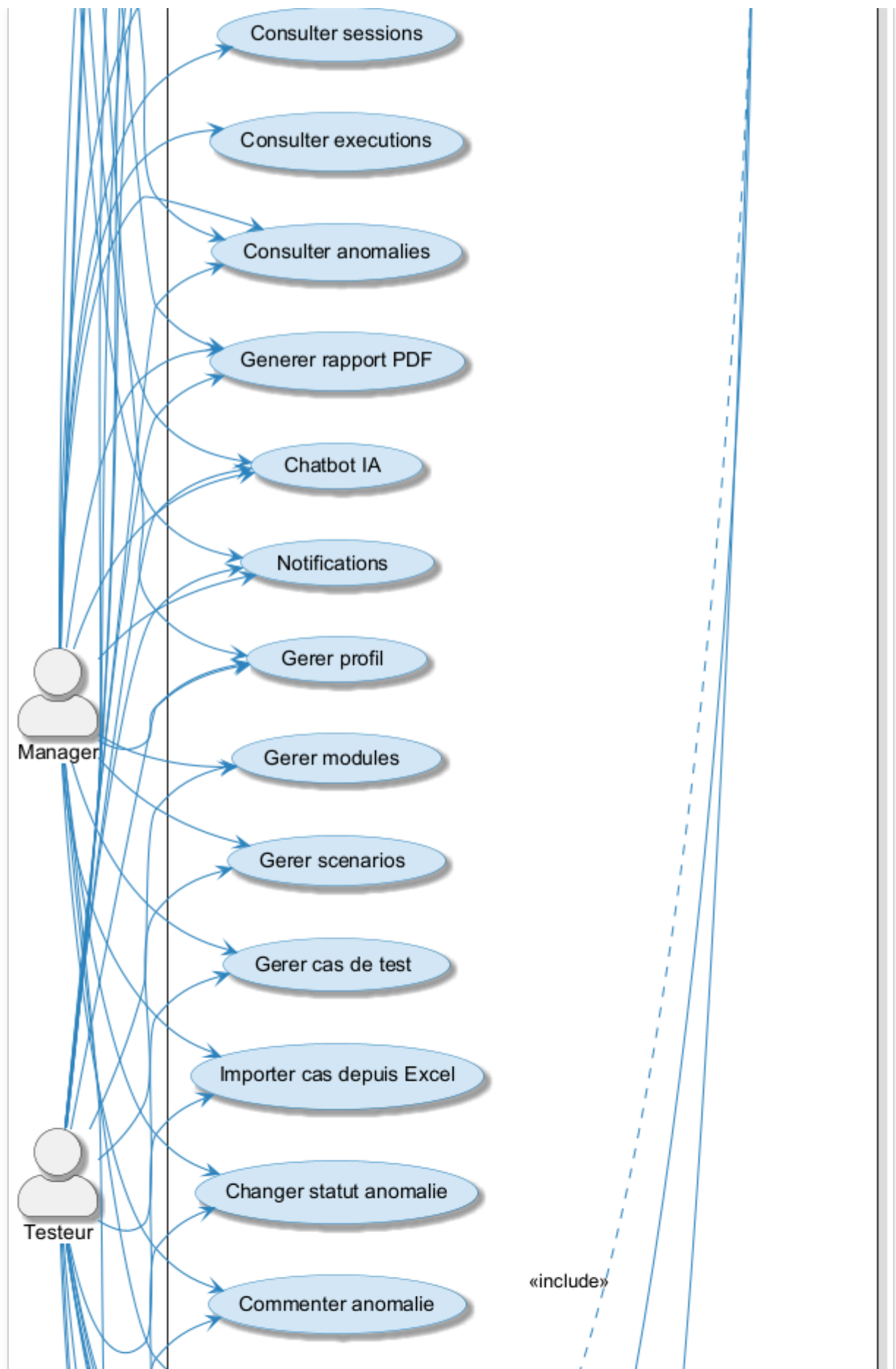
2.1. Vue globale du systeme

Le diagramme de cas d'utilisation global offre une vision d'ensemble de toutes les fonctionnalites accessibles par chacun des trois acteurs du systeme. Il met en evidence la repartition des responsabilites entre l'Admin, le Manager et le Testeur.

L'Admin se concentre sur la gestion des utilisateurs, la creation et l'administration des projets, l'assignation des membres et l'accès au dashboard global. Le Manager intervient dans la validation des projets et la gestion operationnelle des modules, scenarios et cas de test, tout en ayant un accès en consultation aux sessions et executions. Le Testeur est l'acteur principal du processus de test : il cree les sessions, execute les cas de test, declare les anomalies et peut utiliser les fonctionnalites d'intelligence artificielle.

Les relations «include» montrent que certaines fonctionnalites critiques (gestion des utilisateurs, creation de projets, execution, validation) necessitent obligatoirement une authentication prealable.





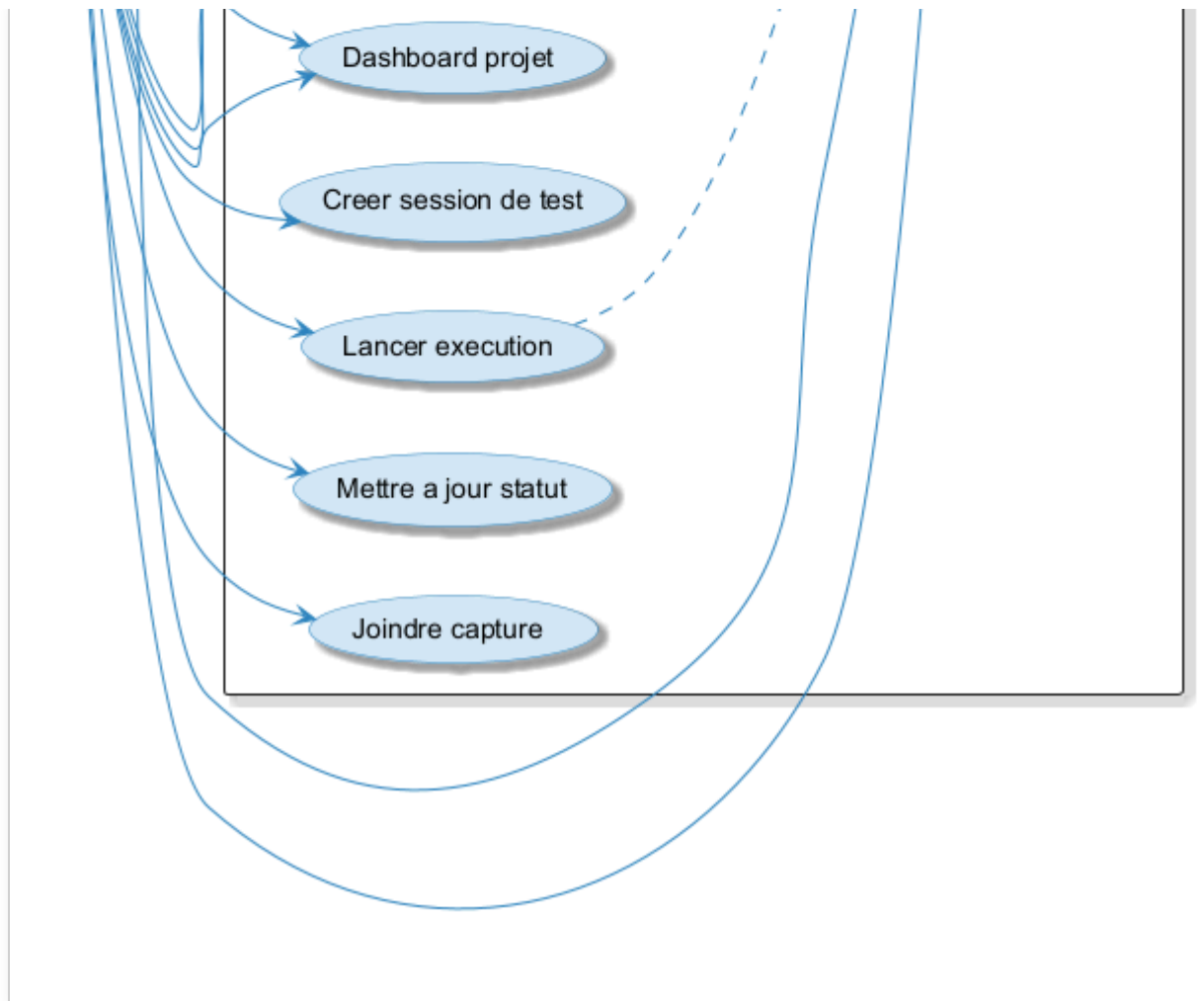


Figure 1 — Diagramme de cas d'utilisation global du système

2.2. Gerer le profil

La gestion du profil est une fonctionnalité commune à tous les acteurs du système. Chaque utilisateur, qu'il soit Admin, Manager ou Testeur, peut accéder à son profil personnel après authentification. Ce cas d'utilisation se décompose en trois sous-cas :

- **Afficher les informations du profil** : l'utilisateur consulte ses données personnelles (nom, prénom, email, rôle).
- **Modifier les informations** : l'utilisateur peut mettre à jour son nom, prénom et autres informations personnelles.
- **Changer le mot de passe** : l'utilisateur peut modifier son mot de passe en fournissant l'ancien et le nouveau mot de passe. Le système vérifie la correspondance avec BCrypt avant d'appliquer le changement.

Ces trois sous-cas sont liés au cas principal par des relations «extend», car ils sont optionnels et déclenchés par choix de l'utilisateur.

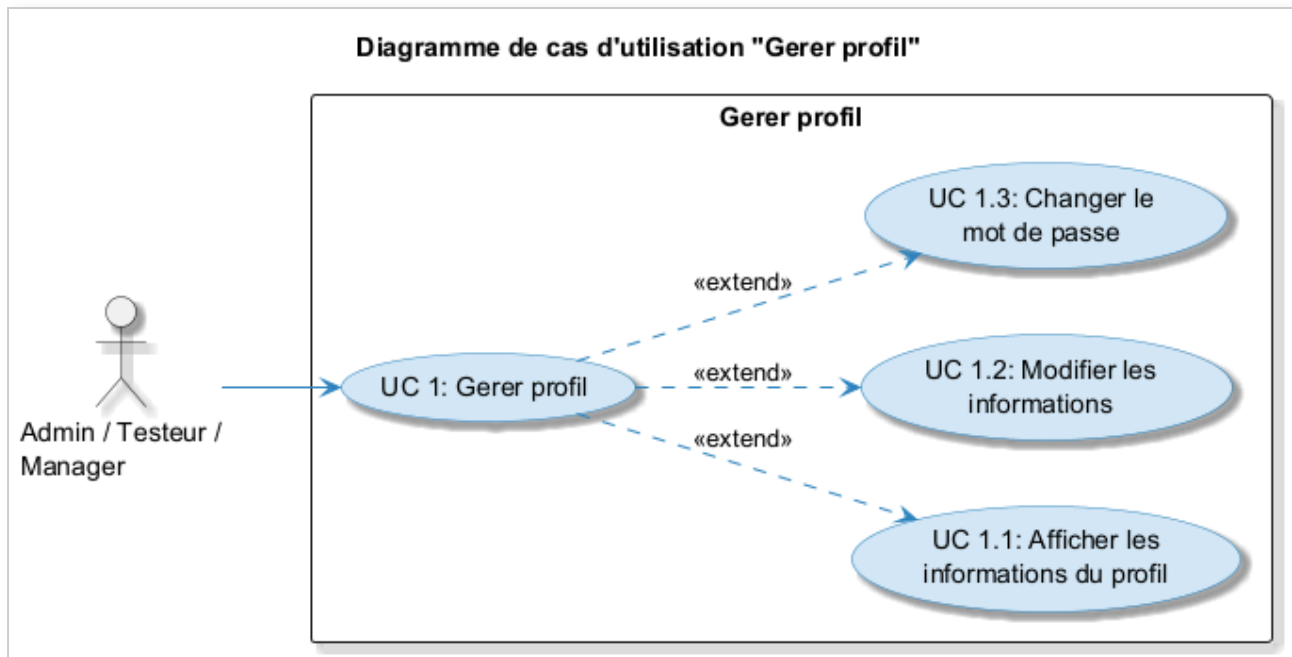


Figure 2 — Diagramme de cas d'utilisation : Gerer le profil

2.3. Gerer les utilisateurs

La gestion des utilisateurs est une fonctionnalité réservée exclusivement à l'Admin. Aucun autre rôle ne dispose des permissions nécessaires pour accéder à ces opérations. Ce module permet à l'administrateur de contrôler l'ensemble des comptes utilisateurs de la plateforme.

Les opérations disponibles sont :

- **Ajouter un utilisateur** : l'Admin crée un nouveau compte en renseignant le nom, prénom, email, nom d'utilisateur, mot de passe et rôle (ADMIN, MANAGER ou TESTEUR).
- **Lister les utilisateurs** : affichage de tous les comptes enregistrés dans le système, avec filtrage possible par rôle ou statut.
- **Modifier un utilisateur** : mise à jour des informations d'un compte existant (nom, email, rôle).
- **Desactiver un utilisateur** : désactivation temporaire d'un compte sans suppression des données associées.
- **Activer un utilisateur** : réactivation d'un compte précédemment désactivé.
- **Supprimer un utilisateur** : suppression définitive d'un compte de la base de données.

- **Lister les testeurs** : filtrage spécifique pour afficher uniquement les utilisateurs ayant le rôle TESTEUR, utile pour l'assignation aux projets.

La relation «include» avec l'authentification garantit que seul un administrateur authentifié peut effectuer ces opérations, conformément aux annotations @PreAuthorize du backend.

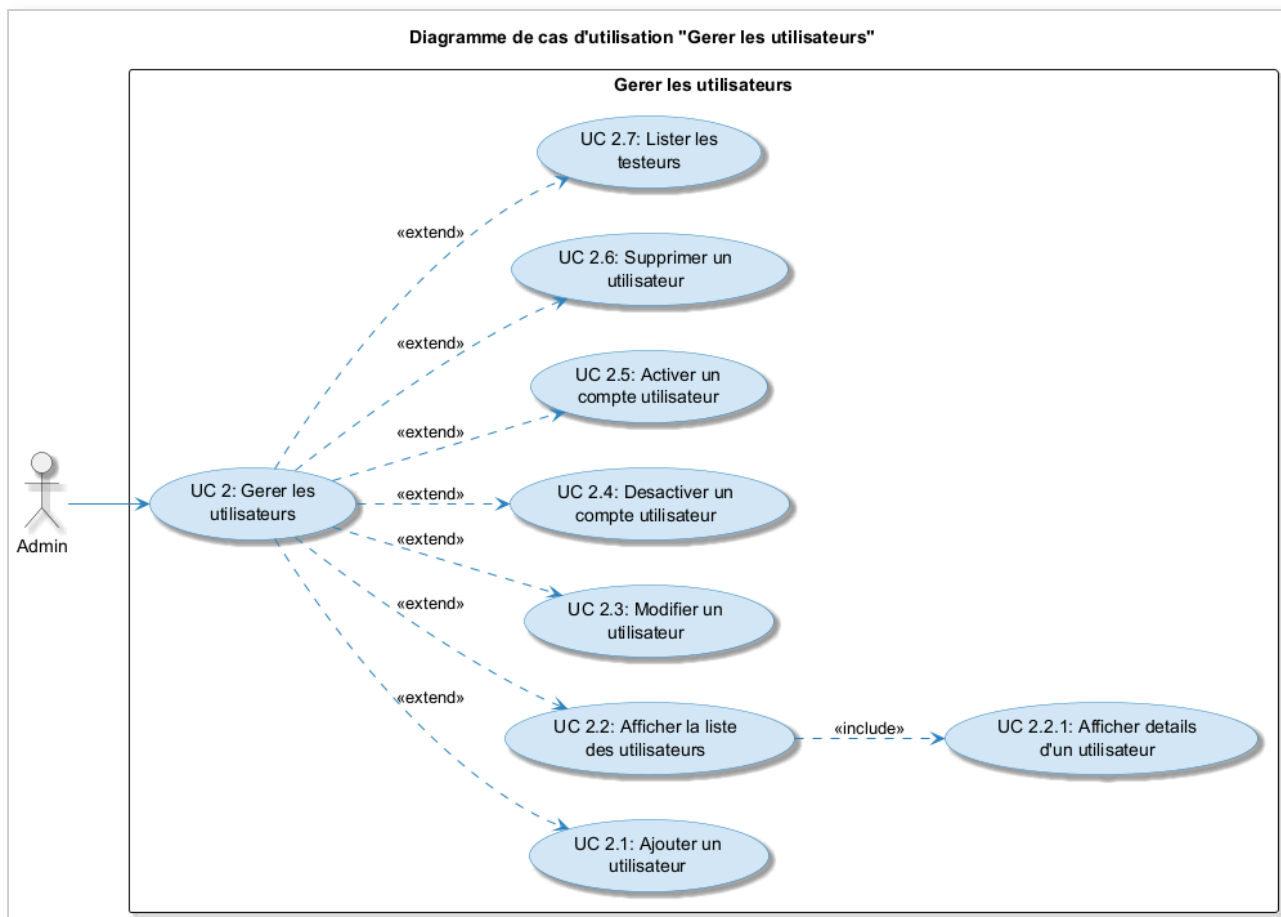


Figure 3 — Diagramme de cas d'utilisation : Gerer les utilisateurs (Admin uniquement)

2.4. Gerer les projets

La gestion des projets est également une prerogative exclusive de l'Admin. Le Manager n'a qu'un droit de consultation et de validation, tandis que le Testeur peut uniquement consulter les projets auxquels il est assigne.

L'Admin dispose des operations suivantes :

- **Creer un projet** : definition du nom, de la description, de la date de debut et de la date de fin prevue. Le statut initial est EN_COURS.
- **Modifier un projet** : mise a jour des informations du projet (nom, description, dates).
- **Supprimer un projet** : suppression en cascade du projet et de toutes ses entites associees (modules, scenarios, cas de test, sessions, executions, anomalies).
- **Archiver un projet** : changement du statut du projet vers ARCHIVE, le rendant en lecture seule.
- **Assigner des membres** : ajout de testeurs et de managers au projet pour qu'ils puissent y travailler.
- **Retirer des membres** : suppression de l'assignation d'un membre a un projet.
- **Consulter les projets** : visualisation de la liste des projets et de leurs details.

Le Manager intervient uniquement pour **valider un projet**, ce qui change le statut de TERMINE a VALIDE, attestant que les tests sont satisfaisants. Le Testeur ne peut que **consulter** les projets auxquels il est affecte.

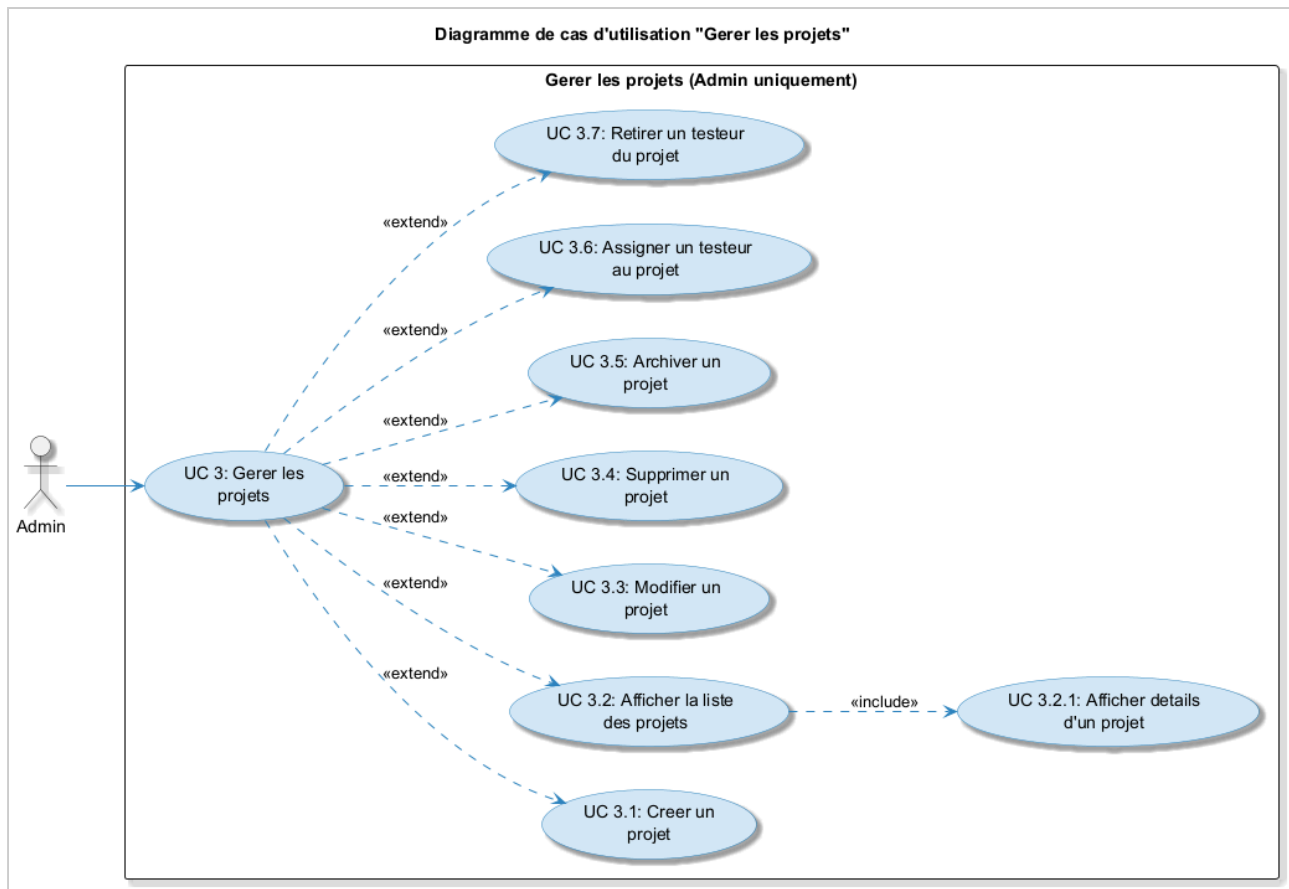


Figure 4 — Diagramme de cas d'utilisation : Gerer les projets

2.5. Gerer les modules

La gestion des modules est accessible au Testeur et au Manager. L'Admin n'intervient pas dans la gestion operationnelle des modules. Un module represente une composante fonctionnelle d'un projet (par exemple : "Module Authentification", "Module Paiement").

Les operations disponibles comprennent :

- **Creer un module** : ajout d'un nouveau module a un projet existant en definissant son nom et sa description.
- **Afficher les modules du projet** : liste de tous les modules appartenant a un projet donne.
- **Modifier un module** : mise a jour du nom ou de la description d'un module.
- **Supprimer un module** : suppression d'un module et de tous ses scenarios associes.
- **Afficher les details d'un module** : sous-cas inclus dans la consultation, permettant de visualiser les informations completes du module et ses scenarios.

La relation «include» entre la consultation et l'affichage des details signifie que la consultation mene systematiquement a la possibilite de voir les details.

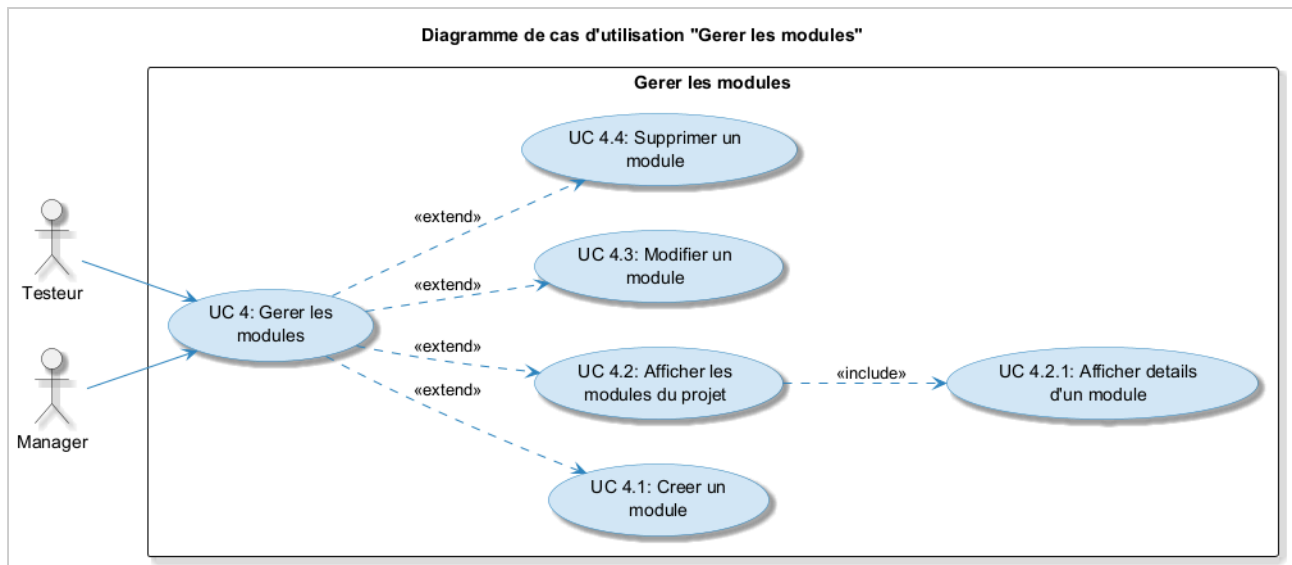


Figure 5 — Diagramme de cas d'utilisation : Gerer les modules

2.6. Gerer les scenarios

Les scenarios representent des parcours fonctionnels a tester au sein d'un module. Comme pour les modules, la gestion des scenarios est accessible au Testeur et au Manager.

Les operations disponibles sont :

- **Creer un scenario** : definition d'un nouveau scenario dans un module, avec un titre et une description du parcours a tester.
- **Afficher les scenarios du module** : liste de tous les scenarios appartenant a un module donne.
- **Modifier un scenario** : mise a jour des informations d'un scenario existant.
- **Supprimer un scenario** : suppression d'un scenario et de tous ses cas de test associes.
- **Afficher les details d'un scenario** : consultation des informations detaillees incluant la liste des cas de test rattaches.

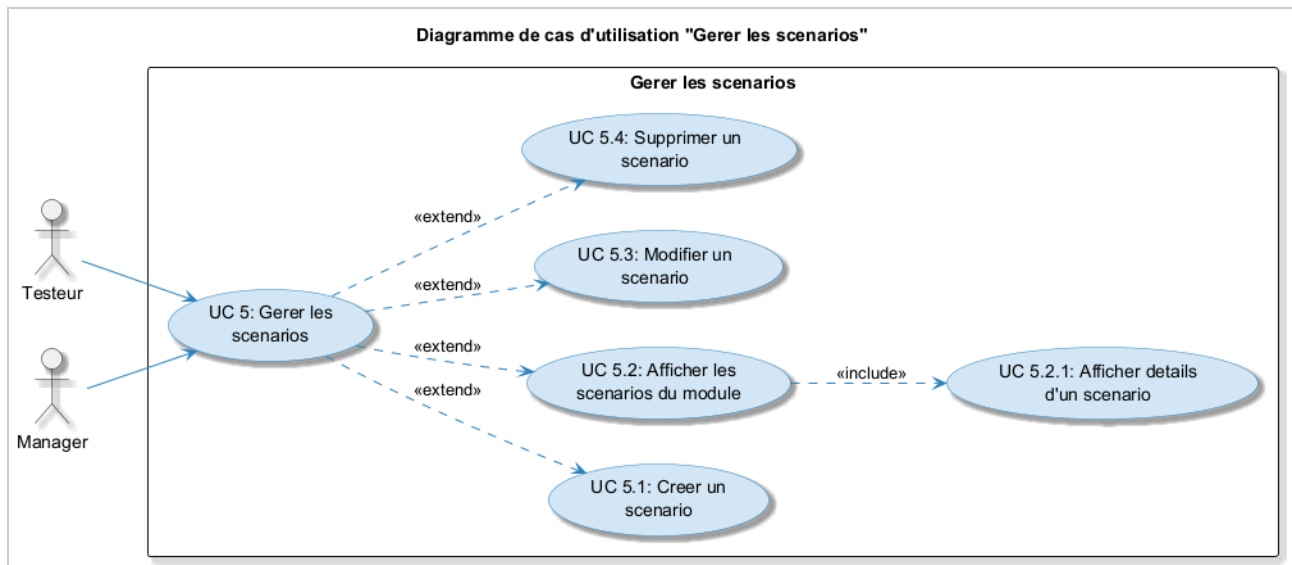


Figure 6 — Diagramme de cas d'utilisation : Gerer les scenarios

2.7. Gerer les cas de test

Les cas de test constituent l'element central du processus de test. Chaque cas de test decrit une procedure precise a executer avec des donnees d'entree, des etapes et un resultat attendu. Le Testeur et le Manager partagent les memes permissions sur cette fonctionnalite.

Les operations disponibles incluent :

- **Creer un cas de test** : saisie manuelle du titre, de la description, des etapes, des donnees d'entree et du resultat attendu.
- **Afficher les cas de test du scenario** : liste de tous les cas de test rattaches a un scenario.
- **Modifier un cas de test** : mise a jour des informations d'un cas existant.
- **Supprimer un cas de test** : suppression d'un cas de test individuel.
- **Importer depuis un fichier Excel** : import en masse de cas de test a partir d'un fichier .xlsx via la bibliotheque Apache POI. Le fichier doit respecter un format predetermine avec les colonnes titre, description, etapes, donnees d'entree et resultat attendu.
- **Generer des cas de test avec l'IA** : utilisation du modele Ollama (deepseek-r1:7b) pour generer automatiquement des cas de test a partir d'une description fonctionnelle. L'utilisateur fournit une description et le nombre souhaite, et l'IA produit des cas de test structures.
- **Afficher les details d'un cas de test** : consultation detaillee d'un cas de test specifique.

L'import Excel et la generation IA sont des extensions optionnelles qui offrent des alternatives productives a la saisie manuelle.

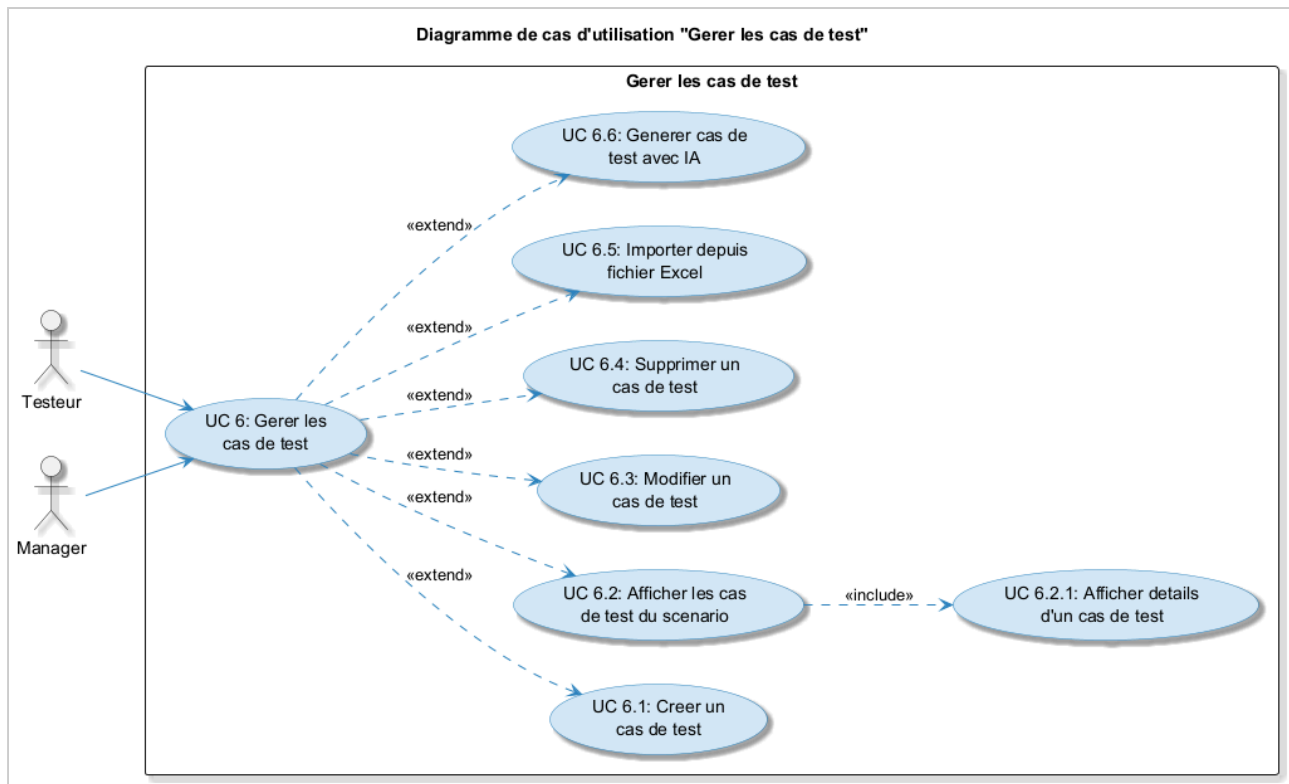


Figure 7 — Diagramme de cas d'utilisation : Gerer les cas de test

2.8. Gerer les sessions de test

Une session de test regroupe un ensemble d'executions dans un contexte temporel defini. La creation et la gestion des sessions sont principalement du ressort du Testeur, tandis que le Manager dispose d'un acces en lecture seule.

Le **Testeur** peut :

- **Creer une session de test** : ouverture d'une nouvelle session avec un statut initial OUVERTE.
- **Modifier une session** : mise a jour des informations de la session (nom, description).
- **Cloturer une session** : fermeture de la session, ce qui declenche automatiquement la generation d'un resume intelligent par l'IA. Ce resume inclut les statistiques d'execution, le taux de reussite et une recommandation go/no-go.

Le **Manager** peut :

- **Consulter les sessions** : visualisation de la liste des sessions d'un projet.
- **Afficher les details d'une session** : consultation des informations detaillees incluant le resume IA genere.

La relation «include» entre la cloture et la generation du resume IA indique que le resume est systematiquement produit lors de chaque cloture de session.

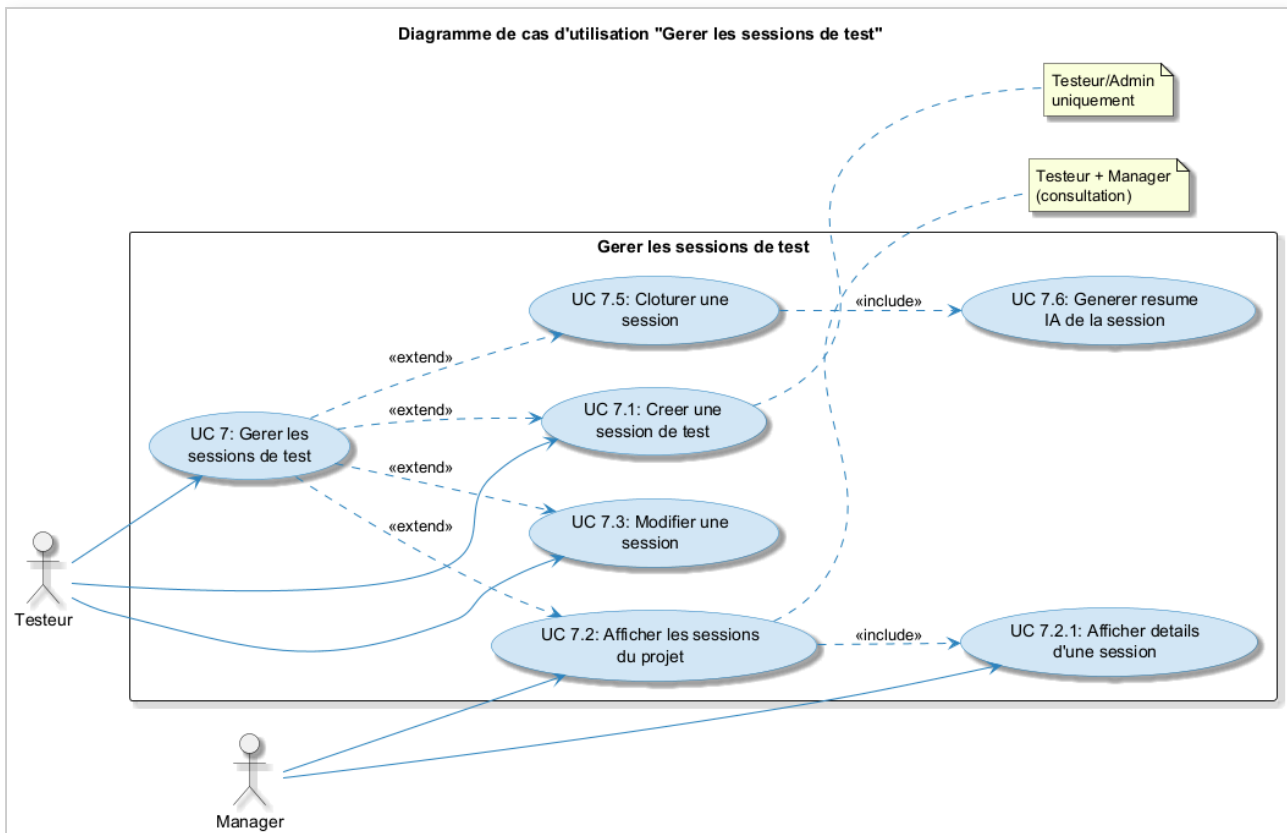


Figure 8 — Diagramme de cas d'utilisation : Gerer les sessions de test

2.9. Gerer les executions

L'execution represente l'action concrete de derouler un cas de test et d'enregistrer son resultat. C'est une activite reservee au Testeur ; le Manager ne dispose que d'un acces en consultation.

Le **Testeur** peut :

- **Lancer une execution** : demarrage de l'execution d'un cas de test dans le cadre d'une session. Le statut initial est PENDING.
- **Mettre a jour le statut** : apres l'execution, le testeur marque le resultat comme SUCCESS (test reussi), FAILED (test echoue) ou BLOCKED (prerequis manquant).
- **Joindre une capture d'ecran** : upload d'une capture d'ecran comme preuve de l'execution via Cloudinary.
- **Afficher les executions** : liste des executions realisees pour un cas de test donne.

Le **Manager** ne peut que consulter les executions et leurs details, sans pouvoir en creer ou en modifier. Cette separation garantit l'integrite du processus de test.

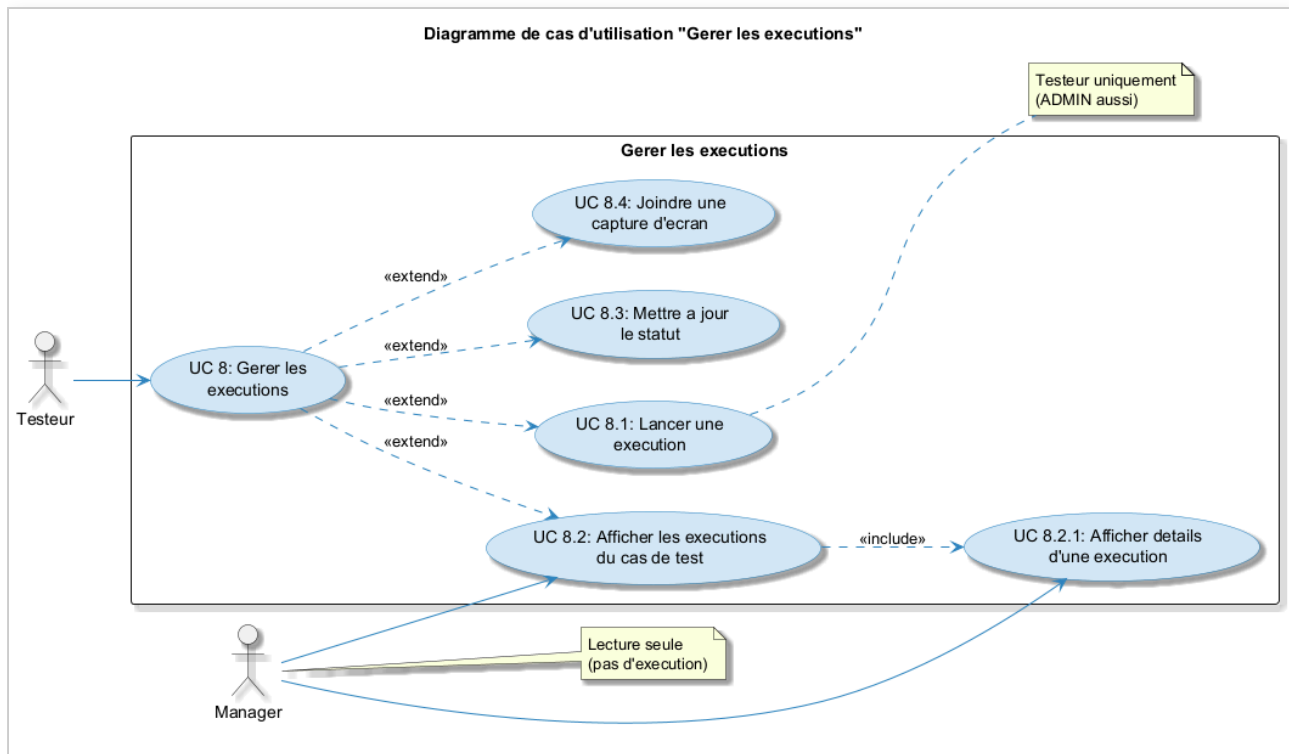


Figure 9 — Diagramme de cas d'utilisation : Gerer les executions

2.10. Gerer les anomalies

La gestion des anomalies est un module critique ou chaque role a des permissions specifiques et distinctes. Une anomalie est creee lorsqu'une execution revele un défaut dans le systeme teste.

Le **Testeur** peut :

- **Declarer une anomalie** : creation d'un rapport d'anomalie avec titre, description, gravite (FAIBLE, MOYENNE, CRITIQUE) et statut initial OUVERTE. Lors de la saisie, l'IA suggere automatiquement un niveau de gravite.
- **Consulter les anomalies** : affichage de la liste et des details des anomalies.
- **Modifier une anomalie** : mise a jour des informations d'une anomalie existante.
- **Changer le statut** : modification du statut (OUVERTE, EN_COURS, CORRIGEE, FERMEE).
- **Commenter une anomalie** : ajout de commentaires pour le suivi.

L'**Admin** peut declarer des anomalies et dispose en plus du droit de **supprimer** une anomalie, operation interdite aux autres roles.

Le **Manager** a un acces restreint : il peut uniquement **consulter** les anomalies, **changer leur statut** et **ajouter des commentaires**. Il ne peut ni declarer ni supprimer d'anomalies.

La relation «include» entre la declaration et la suggestion de gravite IA indique que le systeme propose systematiquement une gravite lors de chaque nouvelle declaration.

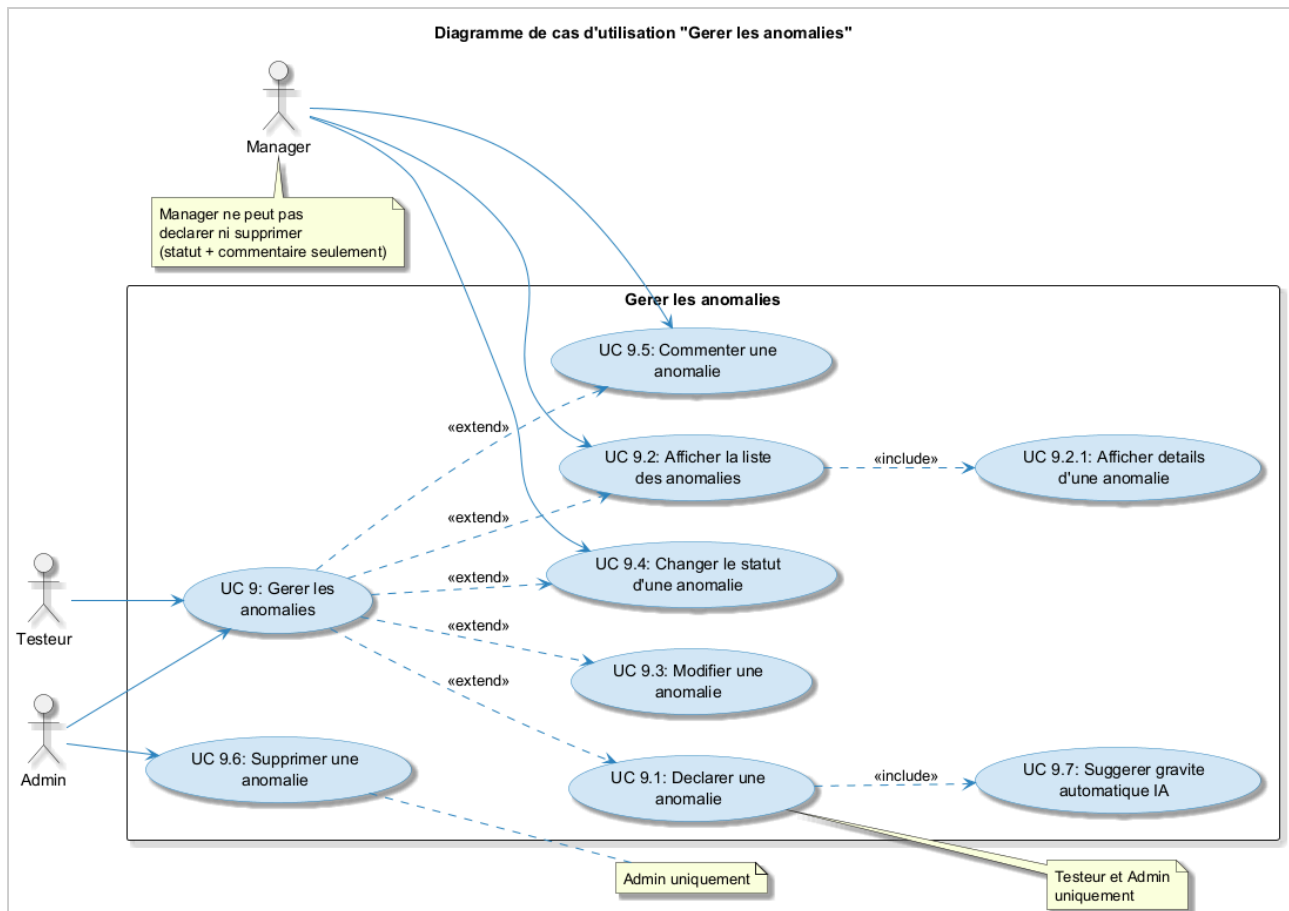


Figure 10 — Diagramme de cas d'utilisation : Gerer les anomalies

2.11. Dashboard et rapports

Le module de tableau de bord et de rapports offre des vues analytiques differenciees selon le role de l'utilisateur.

L'**Admin** dispose de l'acces le plus complet :

- **Dashboard global** : vue d'ensemble de tous les projets avec statistiques agregrees.
- **KPIs globaux** : indicateurs de performance a l'echelle de toute la plateforme.
- **KPIs par projet** : indicateurs detailles pour chaque projet.
- **Generer un rapport PDF** : export des donnees au format PDF via iText 7.
- **Recherche globale** : recherche transversale dans l'ensemble des donnees.
- **Consulter les archives** : acces aux projets archives (fonctionnalite exclusive a l'Admin).

Le **Manager** a acces au dashboard global et aux KPIs globaux, ainsi qu'aux KPIs par projet et a la generation de rapports PDF.

Le **Testeur** dispose d'un acces limite aux KPIs des projets auxquels il est assigne, a la recherche et a la generation de rapports PDF.

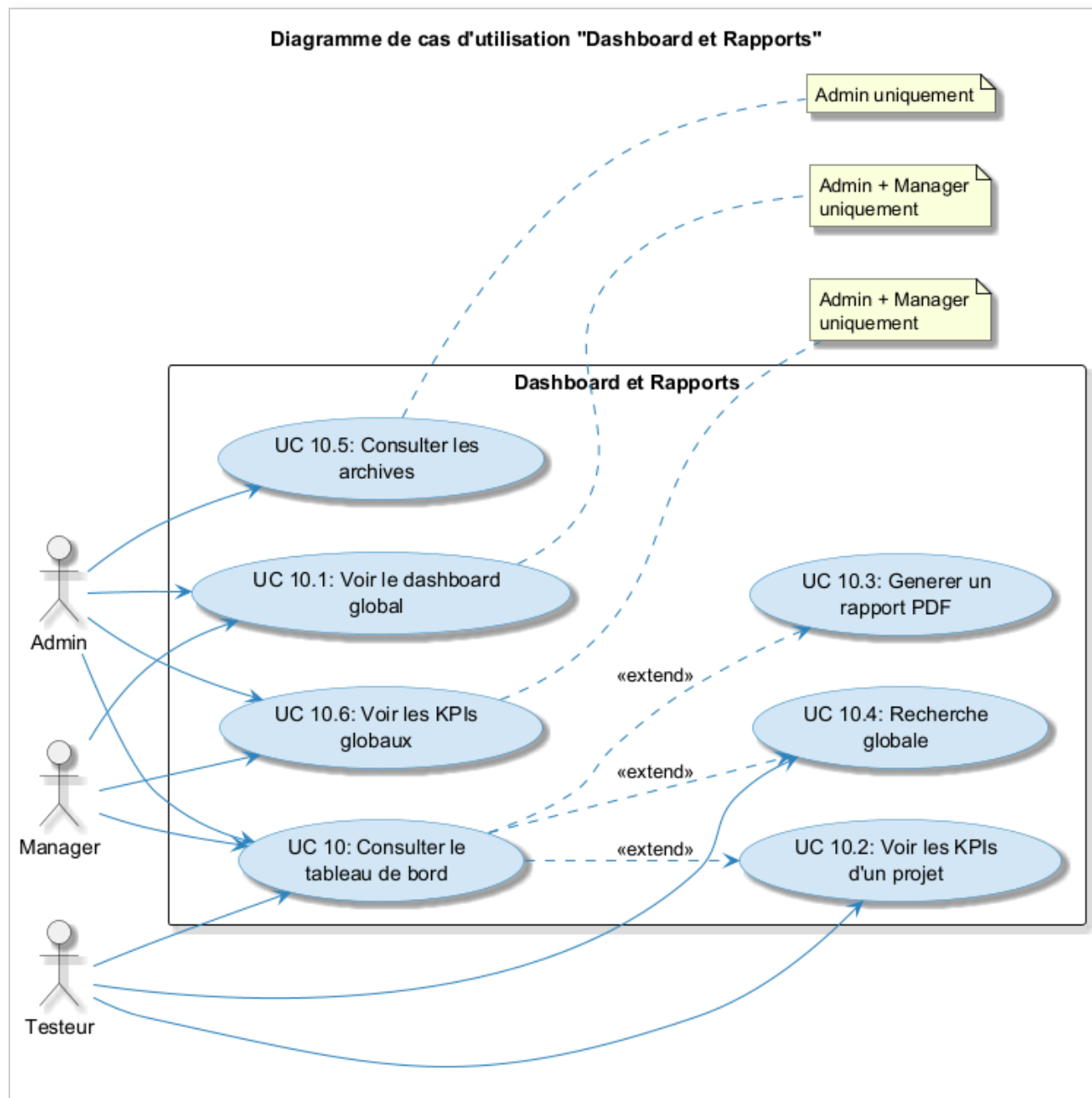


Figure 11 — Diagramme de cas d'utilisation : Dashboard et rapports

2.12. Fonctionnalites d'intelligence artificielle

L'integration de l'intelligence artificielle via Ollama (modele deepseek-r1:7b) constitue une valeur ajoutee significative de la plateforme. Trois acteurs peuvent utiliser ces fonctionnalites, chacune faisant appel au LLM local.

Les fonctionnalités IA comprennent :

- **Generer des cas de test automatiquement** : a partir d'une description fonctionnelle, l'IA produit un ensemble structure de cas de test avec titres, etapes et resultats attendus.
- **Generer un resume intelligent de session** : lors de la cloture d'une session, l'IA analyse les statistiques d'execution et les anomalies pour produire un resume comprehensif incluant une recommandation go/no-go.
- **Obtenir une suggestion de gravite d'anomalie** : lors de la saisie d'une anomalie, l'IA analyse le titre et la description pour suggerer un niveau de gravite (FAIBLE, MOYENNE, CRITIQUE) avec un indice de confiance.
- **Interroger le chatbot IA** : interface conversationnelle permettant de poser des questions libres au modele d'IA.

Toutes ces fonctionnalites incluent obligatoirement une verification de la disponibilite du serveur Ollama («include» vers "Verifier la disponibilite du LLM"), garantissant une gestion gracieuse en cas d'indisponibilite du service IA.

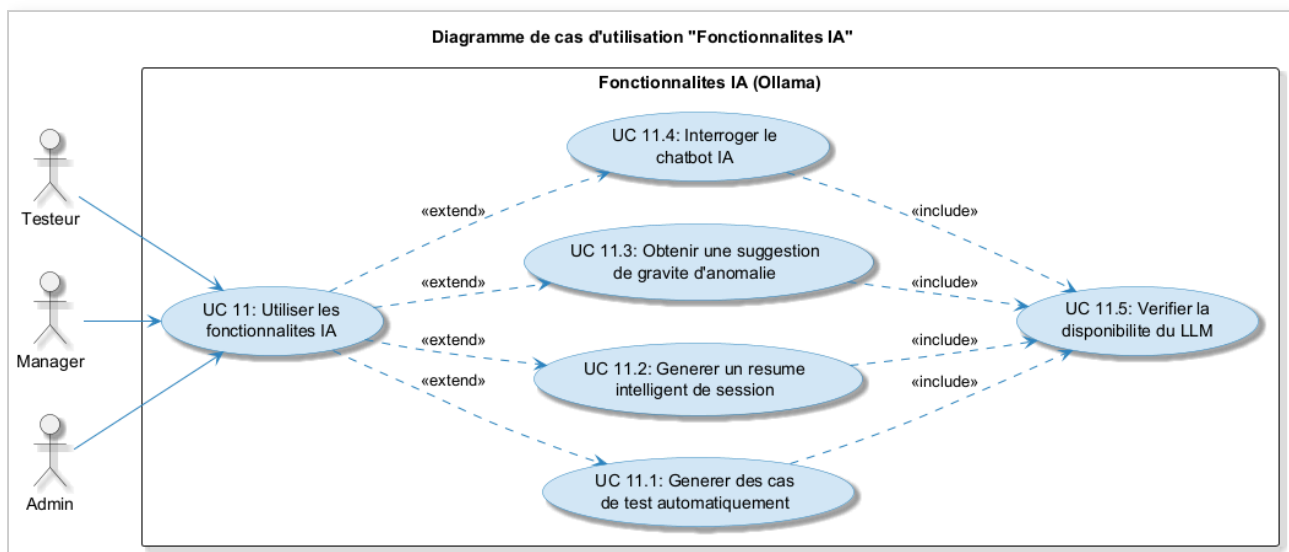


Figure 12 — Diagramme de cas d'utilisation : Fonctionnalites IA

3. Diagramme de classes

Le diagramme de classes represente la structure statique du systeme. Il modelise les entites metier, leurs attributs, leurs methodes et les relations qui les lient. Ce diagramme constitue le socle de la base de donnees relationnelle et du modele de domaine JPA/Hibernate.

3.1. Hierarchie des utilisateurs

La classe abstraite **Utilisateur** constitue la racine de la hierarchie d'heritage. Elle contient les attributs communs (id, username, email, password, nom, prenom, role, actif) et est specialisee en trois sous-classes :

- **Admin** : herite d'Utilisateur avec le role ADMIN. Gere les utilisateurs, les projets et les archives.
- **Manager** : herite d'Utilisateur avec le role MANAGER. Valide les projets et supervise le processus de test.
- **Testeur** : herite d'Utilisateur avec le role TESTEUR. Execute les tests et declare les anomalies.

Cette hierarchie est implementee en JPA via la strategie d'heritage SINGLE_TABLE avec une colonne discriminante "role".

3.2. Chaine de gestion de test

Le coeur du systeme repose sur une chaine hierarchique d'entites :

- **Projet** (1) → (0..*) **ModuleProjet** : un projet contient zero ou plusieurs modules.
- **ModuleProjet** (1) → (0..*) **Scenario** : chaque module contient zero ou plusieurs scenarios.
- **Scenario** (1) → (0..*) **CasDeTest** : chaque scenario contient zero ou plusieurs cas de test.
- **CasDeTest** (1) → (0..*) **Execution** : chaque cas de test peut etre execute plusieurs fois.
- **Execution** (0..1) → (0..1) **Anomalie** : une execution echouee peut generer une anomalie.

3.3. Entites transversales

Plusieurs entites transversales completent le modele :

- **SessionTest** : regroupe un ensemble d'executions dans un contexte temporel. Possede un statut (OUVERTE, CLOTUREE) et un eventuel resume genere par l'IA.

- **Notification** : alerte adresse a un utilisateur, declenchee automatiquement (par exemple lors d'une anomalie critique).
- **Commentaire** : commentaire attache a une anomalie pour le suivi collaboratif.

3.4. Enumerations

Le systeme utilise plusieurs enumerations pour garantir la coherence des donnees :

- **Role** : ADMIN, MANAGER, TESTEUR
- **StatutProjet** : EN_COURS, TERMINE, VALIDE, ARCHIVE
- **StatutExecution** : PENDING, SUCCESS, FAILED, BLOCKED
- **StatutAnomalie** : OUVERTE, EN_COURS, CORRIGEE, FERMEE
- **Gravite** : FAIBLE, MOYENNE, CRITIQUE

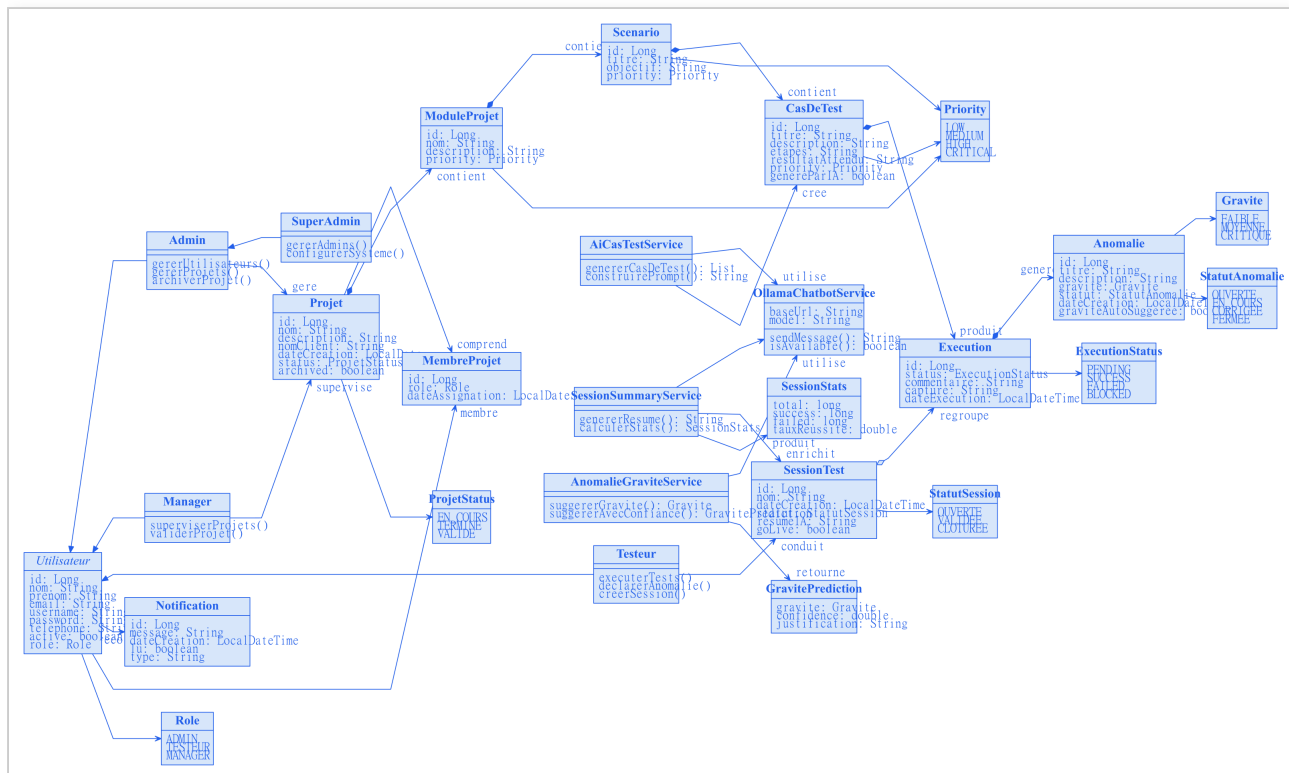


Figure 13 — Diagramme de classes du systeme

4. Diagrammes de sequence

Les diagrammes de sequence modelisent les interactions dynamiques entre les acteurs et les composants du systeme dans le temps. Ils decrivent l'ordre des messages echanges lors de l'execution des cas d'utilisation les plus significatifs.

4.1. Sequence : Authentification

Le processus d'authentification est le point d'entree de toute interaction avec le systeme. Il implique les composants suivants : l'utilisateur, le frontend React, le AuthController, le AuthService, le JwtService et la base de donnees.

Le flux se deroule comme suit :

1. L'utilisateur saisit son nom d'utilisateur et son mot de passe dans le formulaire de connexion.
2. Le frontend envoie une requete POST a l'endpoint `/api/auth/login`.
3. Le AuthController delegue au AuthService qui recherche l'utilisateur en base de donnees.
4. Le mot de passe est verifie via BCrypt.
5. **Cas d'echec** : si le mot de passe est invalide, une exception `BadCredentialsException` est levee, et le frontend affiche un message d'erreur (401 Unauthorized).
6. **Cas de succes** : le JwtService genere un token JWT valable 24 heures. Le token, le role et l'identifiant de l'utilisateur sont renvoyes au frontend, qui stocke le token dans le `localStorage` et redirige vers le dashboard.

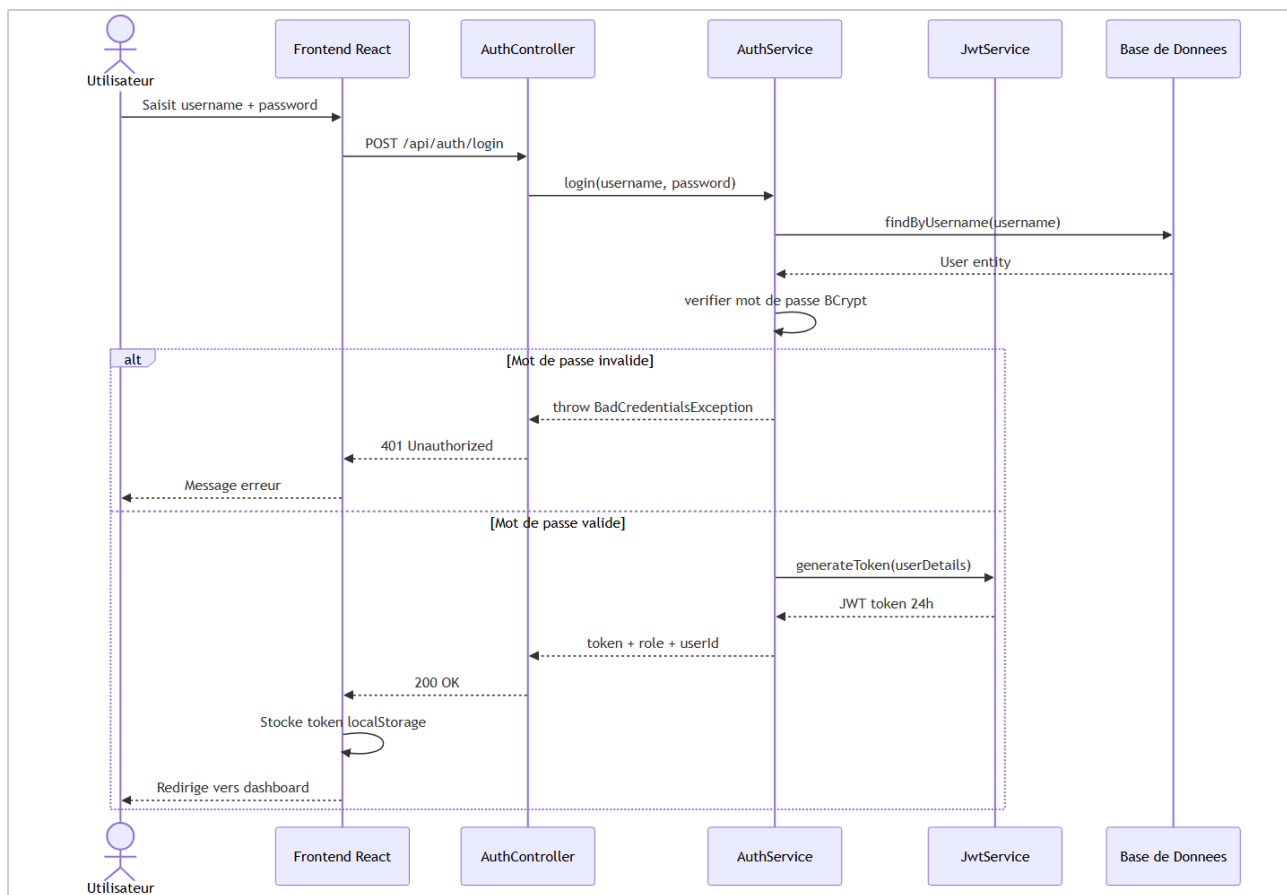


Figure 14 — Diagramme de sequence : Authentification

4.2. Sequence : Processus d'exécution de test

Ce diagramme illustre le flux complet d'un cycle de test, depuis la création de la session jusqu'à la déclaration d'une anomalie. Il montre comment un testeur interagit avec plusieurs contrôleurs pour mener à bien le processus.

Le flux se déroule en quatre phases :

1. **Création de la session** : le testeur crée une session de test via `POST /api/sessions`. La session est initialisée avec le statut `OUVERTE`.
2. **Lancement de l'exécution** : le testeur lance l'exécution d'un cas de test via `POST /api/executions`. L'exécution est créée avec le statut `PENDING`.
3. **Mise à jour du statut** : après avoir exécuté le test, le testeur met à jour le statut (ici `FAILED`) via `PATCH /api/executions/id`.
4. **Déclaration d'anomalie** : suite à l'échec, le testeur déclare une anomalie critique via `POST /api/anomalies`. Le système enregistre l'anomalie avec le statut `OUVERTE` et déclenche automatiquement des notifications aux Admin et Manager via le `NotificationService`.

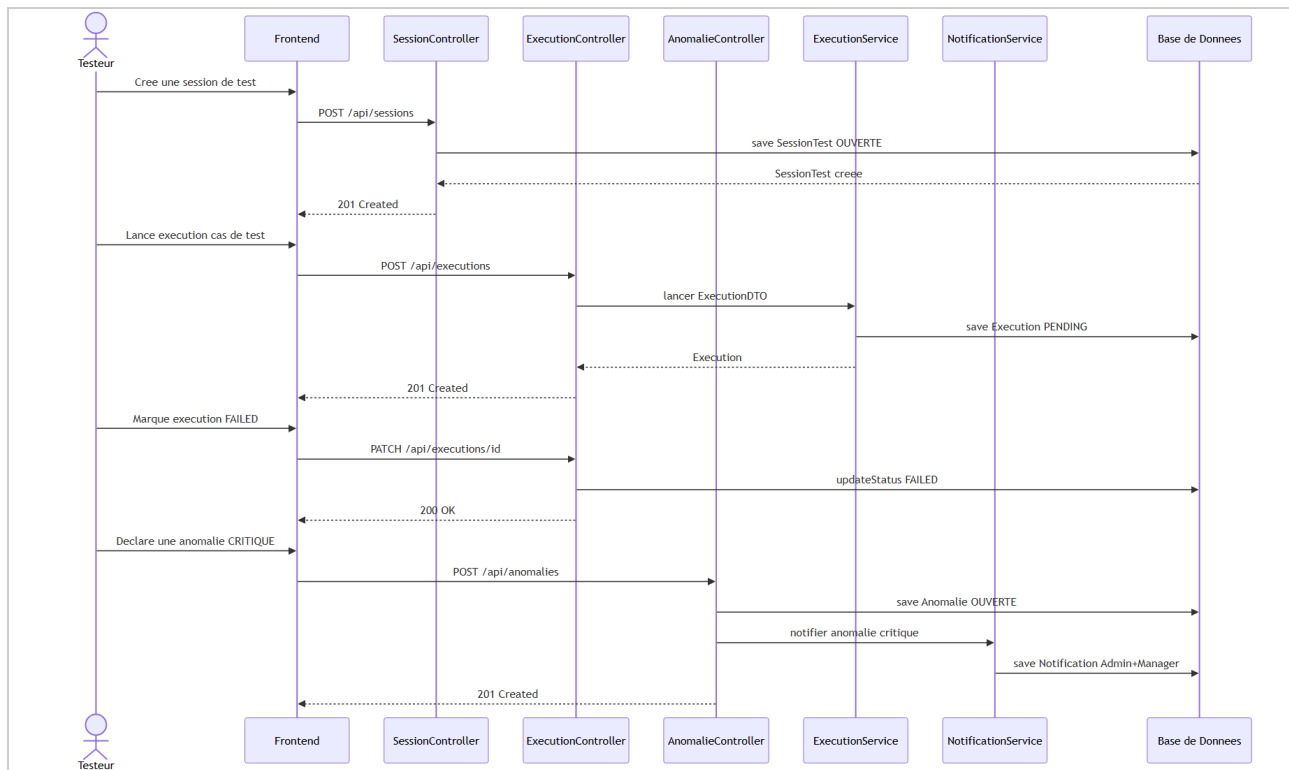


Figure 15 — Diagramme de sequence : Processus d'exécution de test

4.3. Sequence : Generation de cas de test par IA

Ce diagramme detaille le processus de generation automatique de cas de test via l'intelligence artificielle. C'est l'une des fonctionnalites les plus innovantes de la plateforme.

Le flux se deroule comme suit :

1. Le testeur clique sur le bouton "Generer avec IA" et saisit une description fonctionnelle ainsi que le nombre de cas de test souhaite.
2. Le frontend envoie une requete POST a `/api/ai/generer-cas-tests` avec la description, le nombre et l'identifiant du scenario.
3. Le AiController delegue au AiCasTestService qui recupere d'abord le scenario en base de donnees pour obtenir le contexte.
4. Le service construit un prompt contextualise incluant les informations du module et du scenario.
5. Le prompt est envoye au OllamaChatbotService qui communique avec le serveur Ollama local via `POST /api/generate`.
6. Le LLM renvoie une reponse JSON contenant les cas de test generes.

7. Le service parse la reponse JSON et, dans une boucle, sauvegarde chaque cas de test en base de donnees avec le flag genereParIA=true.
8. La liste des cas de test crees est renvoyee au frontend pour affichage.

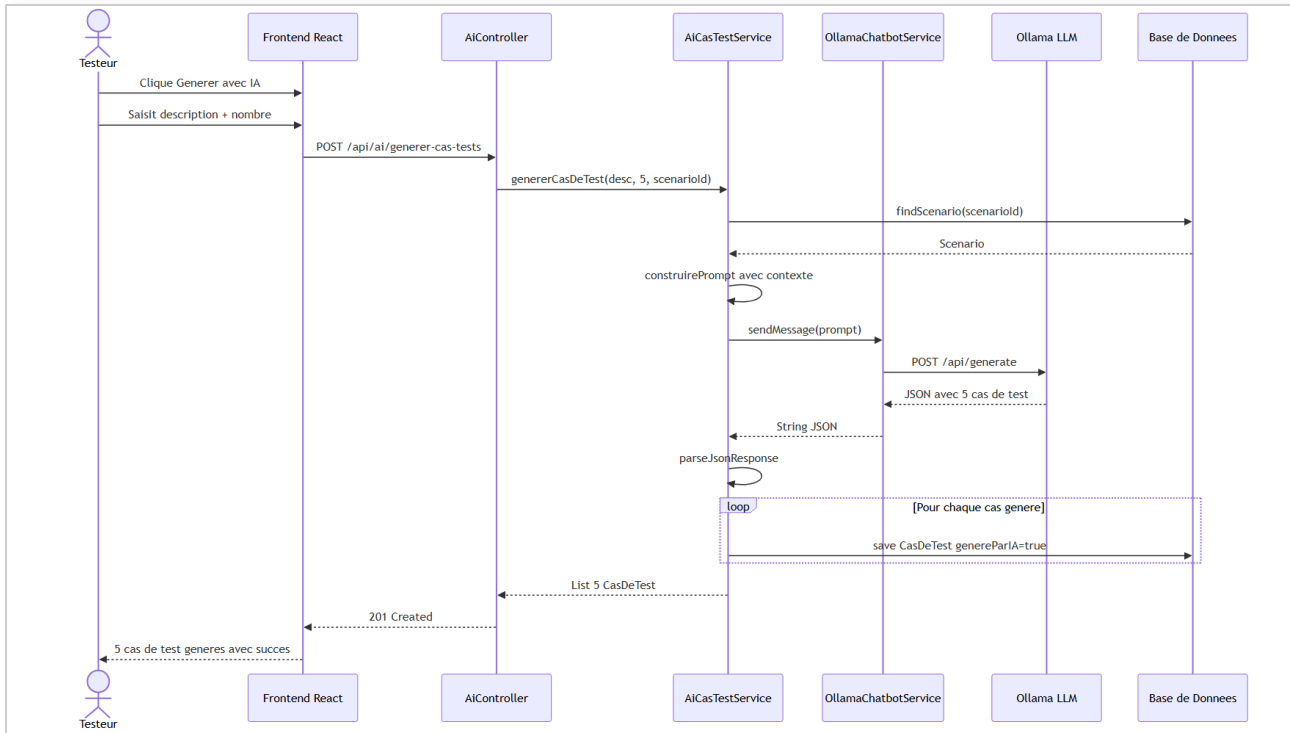


Figure 16 — Diagramme de sequence : Generation de cas de test par IA

4.4. Sequence : Resume intelligent de session

Ce diagramme illustre le processus de generation automatique d'un resume de session de test lors de sa cloture. Le resume inclut des statistiques detaillees et une recommandation go/no-go.

Le flux se deroule comme suit :

1. Le manager clique sur "Cloturer la session", declenchant une requete PATCH vers `/api/sessions/id/cloturer`.
2. Le SessionTestService recupere toutes les executions et anomalies de la session en base de donnees.
3. Le SessionSummaryService calcule les statistiques (taux de succes, nombre d'anomalies par gravite, etc.).
4. Un prompt est construit avec ces statistiques et envoye au LLM via Ollama.
5. Le LLM genere un resume en francais avec une analyse des resultats et une recommandation.
6. Le resume est sauvegarde dans l'entite SessionTest et le statut passe a CLOTUREE.
7. Le resume et la recommandation go/no-go sont renvoyes au frontend pour affichage.

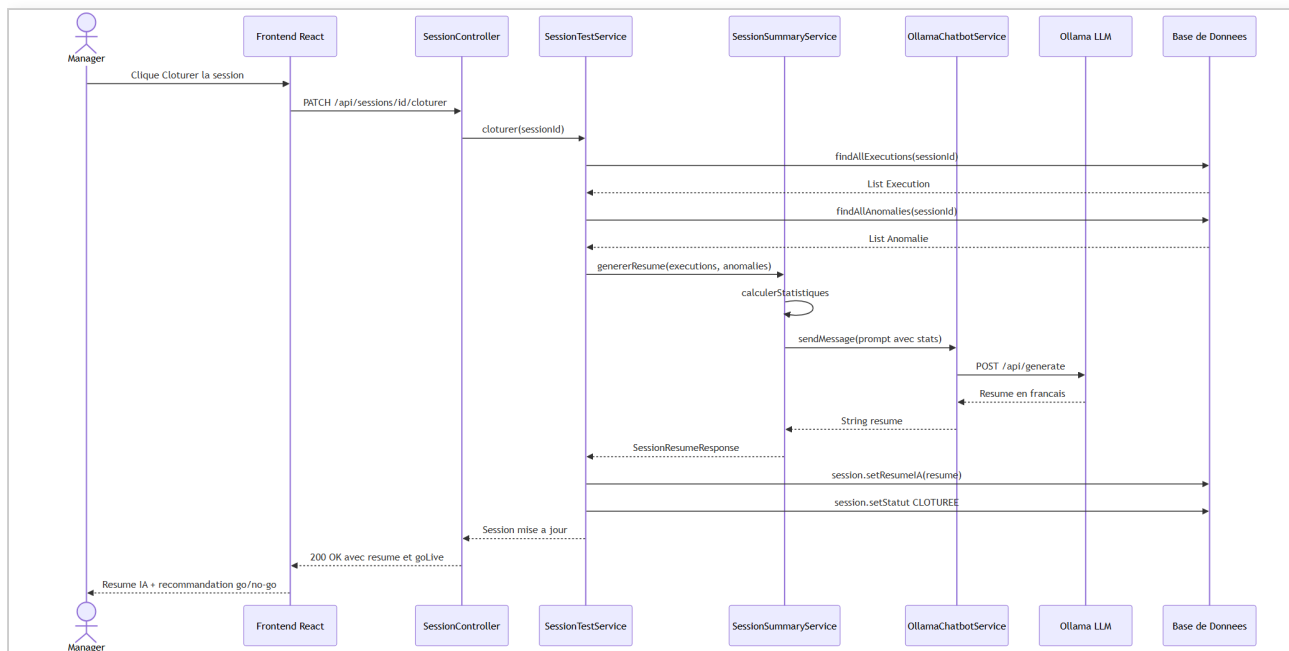


Figure 17 — Diagramme de sequence : Resume intelligent de session par IA

4.5. Sequence : Suggestion de gravite par IA

Ce diagramme montre comment l'IA assiste le testeur en suggérant automatiquement le niveau de gravité d'une anomalie lors de sa saisie.

Le flux se deroule comme suit :

1. Le testeur commence a saisir le titre et la description de l'anomalie dans le formulaire.
2. Apres un debounce de 800 millisecondes (pour eviter les appels excessifs), le frontend envoie une requete POST a `/api/ai/suggerer-gravite`.
3. Le AnomalieGraviteService construit un prompt et l'envoi au LLM.
4. Le LLM analyse le contenu et renvoie une suggestion (par exemple "CRITIQUE - perte de donnees").
5. Le service parse la reponse et calcule un indice de confiance (par exemple 92%).
6. La suggestion est affichee sous forme de badge dans le formulaire : "IA suggere : CRITIQUE (92%)".
7. Le testeur peut accepter ou modifier la suggestion avant de soumettre le formulaire.
8. L'anomalie est finalement enregistree en base de donnees via le AnomalieController.

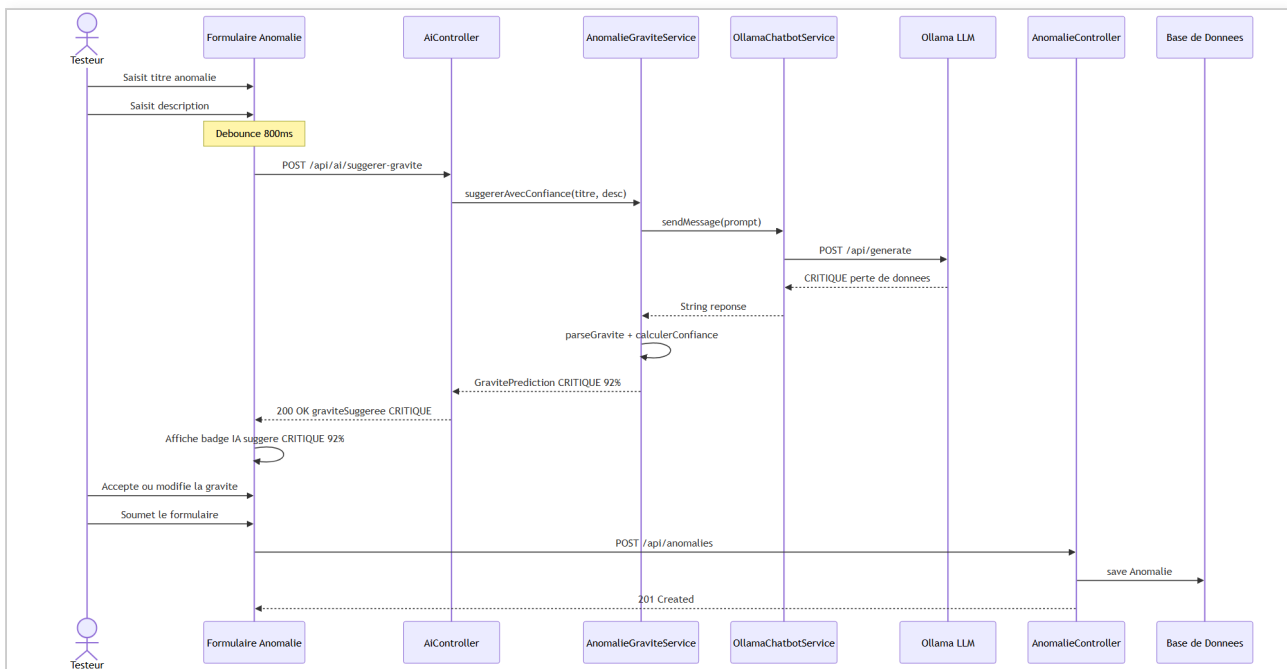


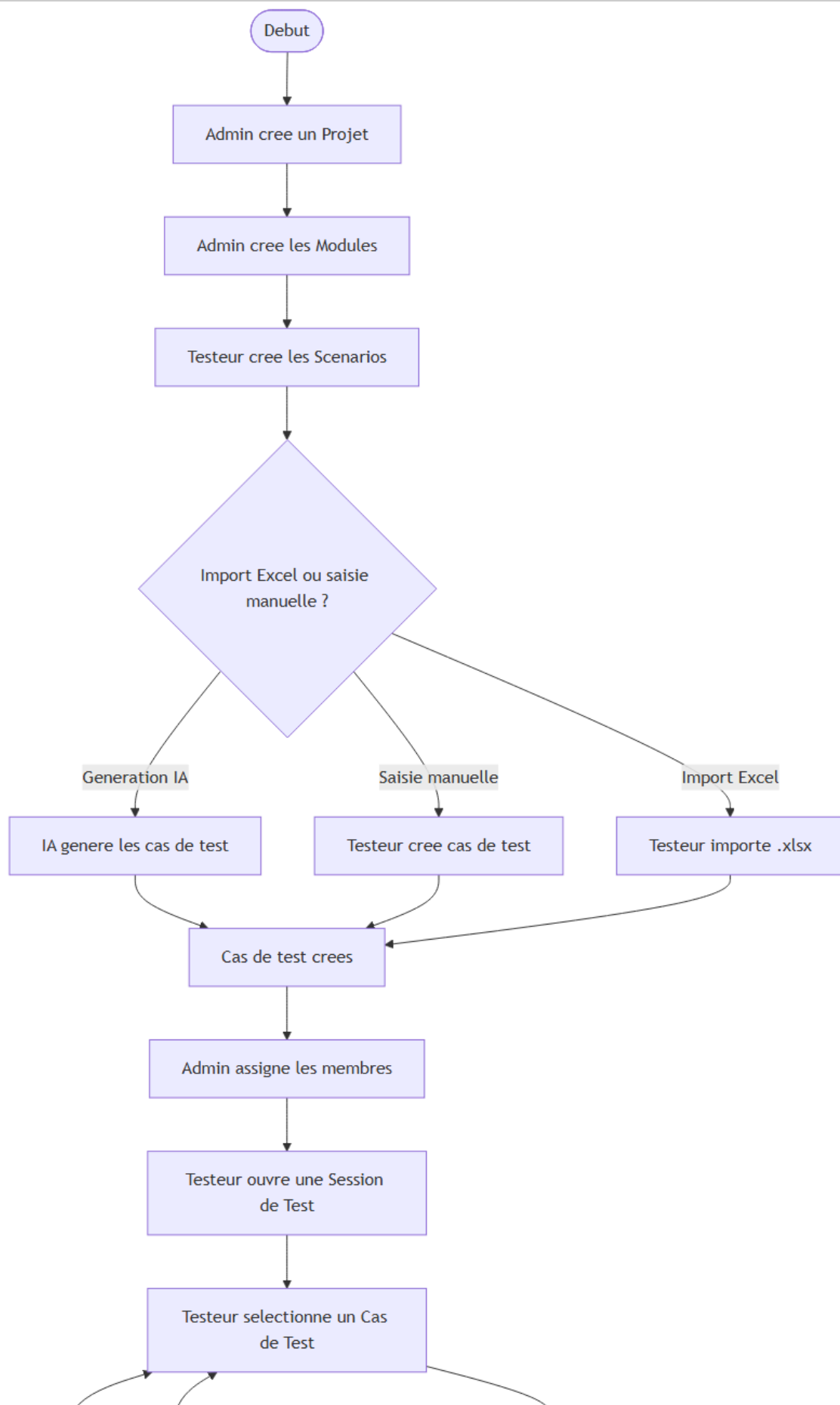
Figure 18 — Diagramme de sequence : Suggestion de gravite par IA

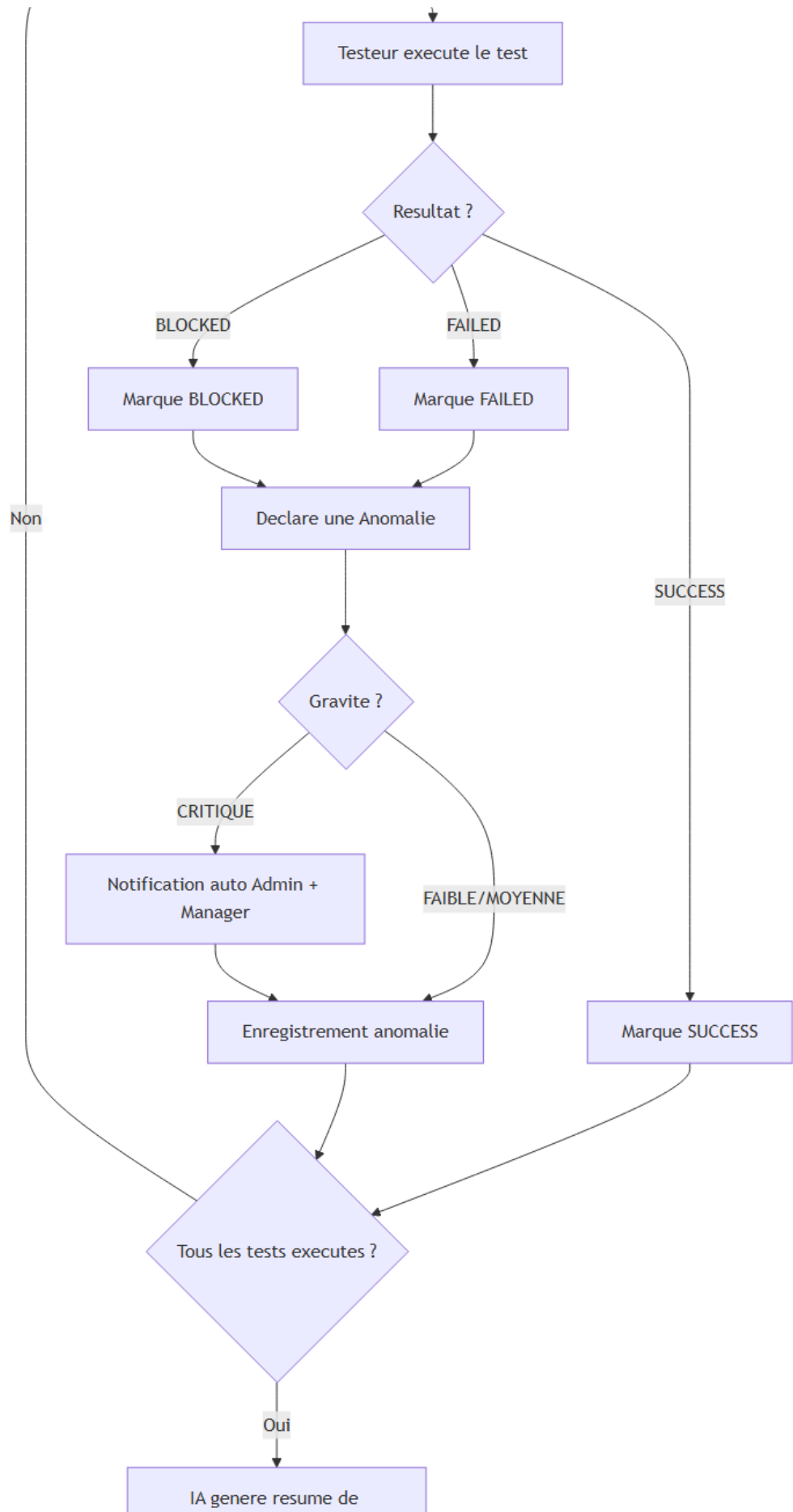
5. Diagramme d'activite

Le diagramme d'activite modelise le flux de travail global du processus de gestion de test, depuis la creation d'un projet jusqu'a son archivage. Il met en evidence les points de decision et les chemins alternatifs.

Le processus se deroule en sept phases principales :

1. **Initialisation du projet** : l'Admin cree un projet, puis cree les modules qui le composent.
2. **Preparation des cas de test** : le Testeur cree les scenarios dans chaque module, puis alimente les cas de test. Un point de decision offre trois alternatives : la saisie manuelle, l'import depuis un fichier Excel ou la generation automatique par IA.
3. **Assignment** : l'Admin assigne les membres (testeurs et managers) au projet.
4. **Execution des tests** : le Testeur ouvre une session de test, selectionne un cas de test et l'execute. Un point de decision determine le resultat : SUCCESS, FAILED ou BLOCKED.
5. **Gestion des anomalies** : en cas d'echec ou de blocage, le testeur declare une anomalie. Si la gravite est CRITIQUE, des notifications automatiques sont envoyees aux Admin et Manager.
6. **Bilan** : une fois tous les tests executes, l'IA genere un resume de session. Le Manager consulte les resultats et decide : si les resultats sont satisfaisants, il valide le projet ; sinon, il demande des corrections et le cycle de test reprend.
7. **Archivage** : apres validation, l'Admin archive le projet, marquant la fin du processus.





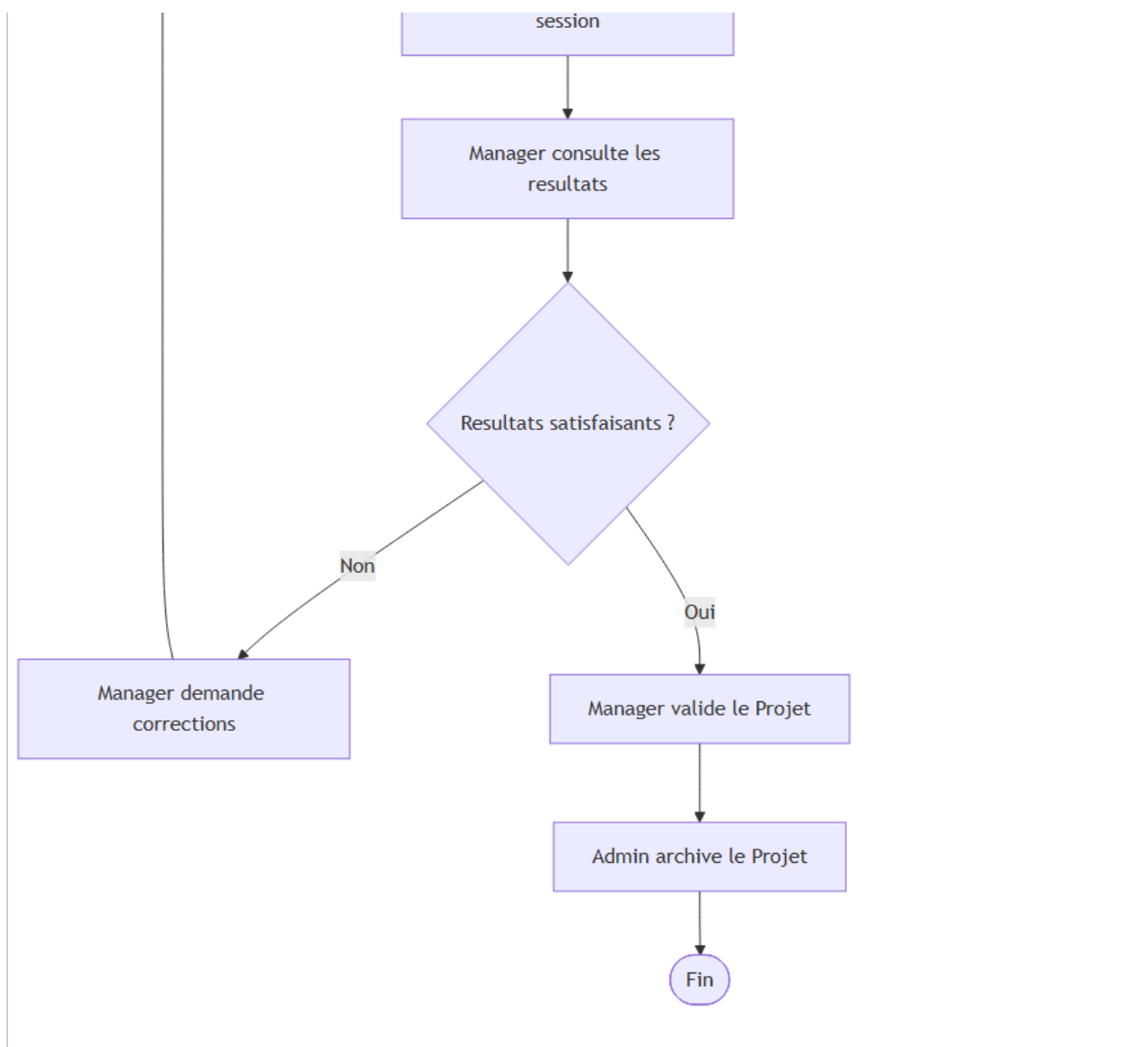


Figure 19 — Diagramme d'activité : Processus global de gestion de test

6. Diagrammes d'etats-transitions

Les diagrammes d'etats-transitions modelisent les differents etats possibles d'une entite et les evenements qui provoquent les transitions entre ces etats. Nous presentons trois diagrammes pour les entites principales : Projet, Anomalie et Execution.

6.1. Etats d'un projet

Un projet traverse quatre etats au cours de son cycle de vie :

Etat	Description	Transitions possibles
EN_COURS	Etat initial apres la creation par l'Admin. Les tests sont en cours d'execution.	TERMINE (tests executes) ou ARCHIVE (archivage direct)
TERMINE	Tous les tests prevus ont ete executes.	VALIDE (Manager valide), EN_COURS (Manager refuse, retour aux corrections) ou ARCHIVE
VALIDE	Le Manager a atteste que les resultats de test sont satisfaisants.	ARCHIVE (archivage par l'Admin)
ARCHIVE	Etat terminal. Le projet est en lecture seule, accessible uniquement par l'Admin.	Aucune (etat final)

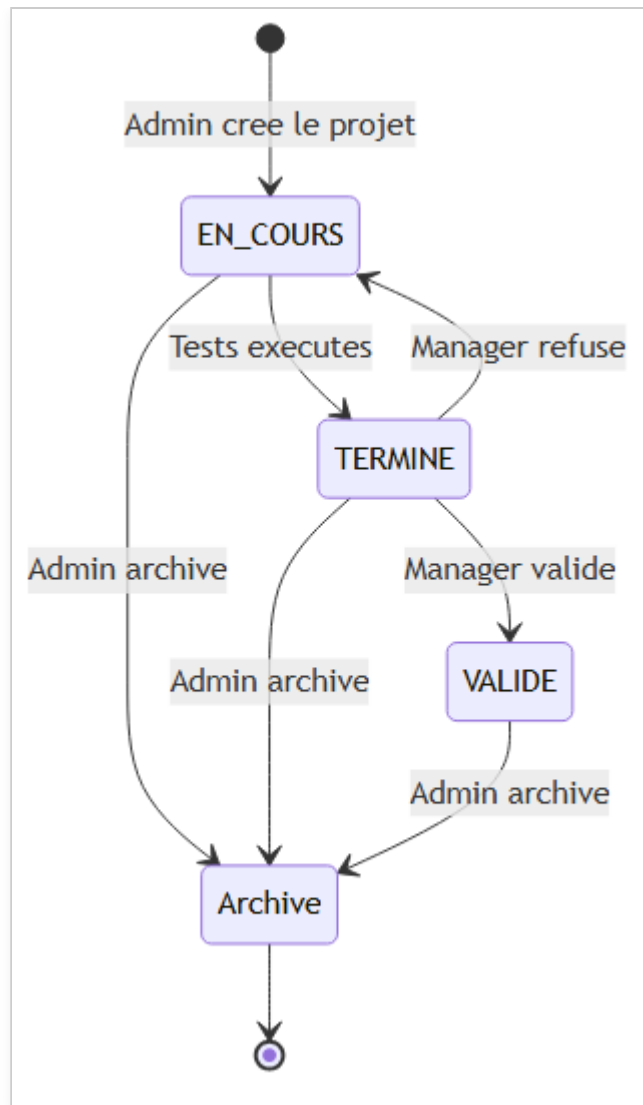


Figure 20 — Diagramme d'etats-transitions : Projet

6.2. Etats d'une anomalie

Une anomalie suit un cycle de vie en quatre etats :

Etat	Description	Transitions possibles
OUVERTE	Etat initial. L'anomalie vient d'etre declaree par le Testeur ou l'Admin.	EN_COURS (prise en charge) ou FERMEE (faux positif)
EN_COURS	L'anomalie est en cours de traitement ou de correction.	CORRIGEE (correction appliquee) ou OUVERTE (remise en attente)
CORRIGEE	La correction a ete appliquee, en attente de validation.	FERMEE (Manager valide la correction) ou EN_COURS (correction insuffisante)
FERMEE	Etat terminal. L'anomalie est resolue ou classee.	Aucune (etat final)

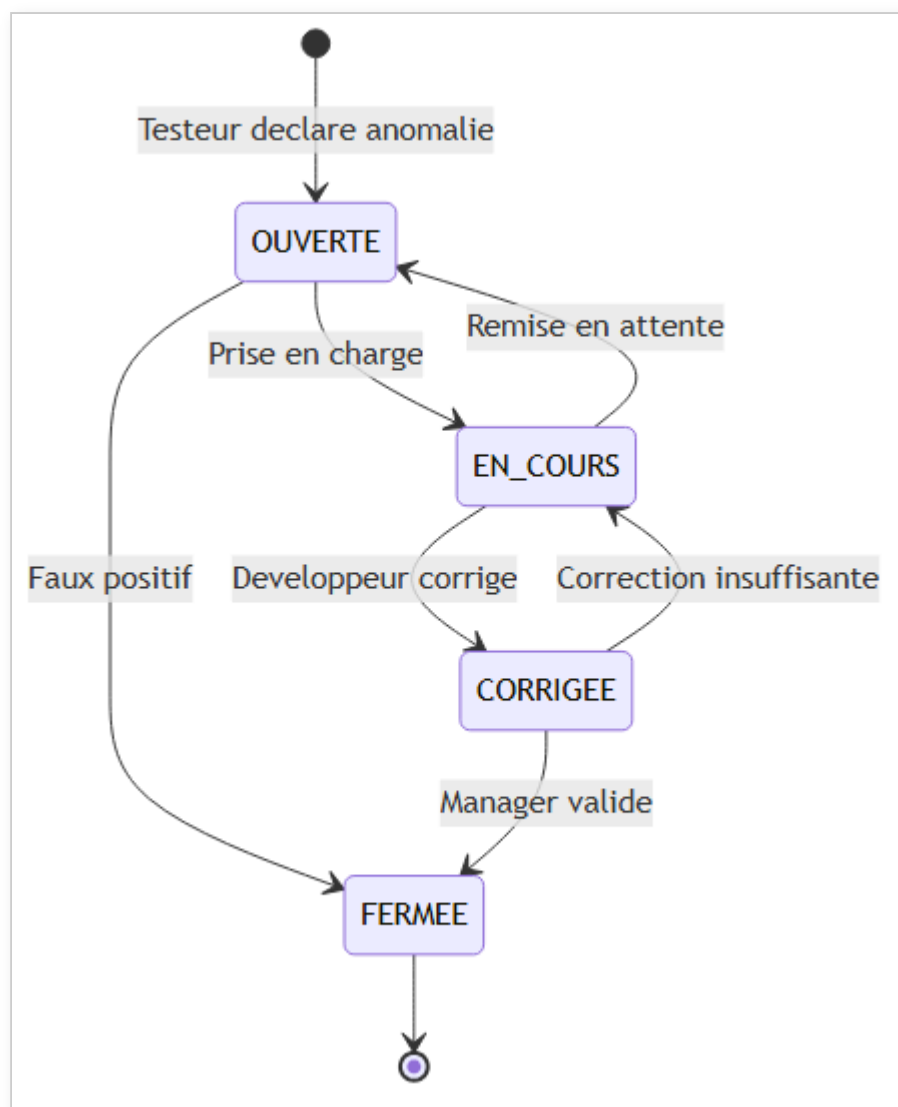


Figure 21 — Diagramme d'etats-transitions : Anomalie

6.3. Etats d'une execution

Une execution possede quatre etats possibles :

Etat	Description	Transitions possibles
PENDING	Etat initial. L'execution a ete lancee mais pas encore completee.	SUCCESS, FAILED ou BLOCKED
SUCCESS	Le test a ete execute avec succes.	FAILED (re-execution echouee) ou etat final (session cloturee)
FAILED	Le test a echoue. Une anomalie peut etre declaree.	SUCCESS (correction reussie) ou etat final
BLOCKED	Le test ne peut pas etre execute (prerequis manquant).	PENDING (prerequis resolu) ou etat final

Les transitions entre SUCCESS et FAILED permettent de re-executer un test apres correction. Toutes les executions atteignent leur etat final lorsque la session est cloturee.

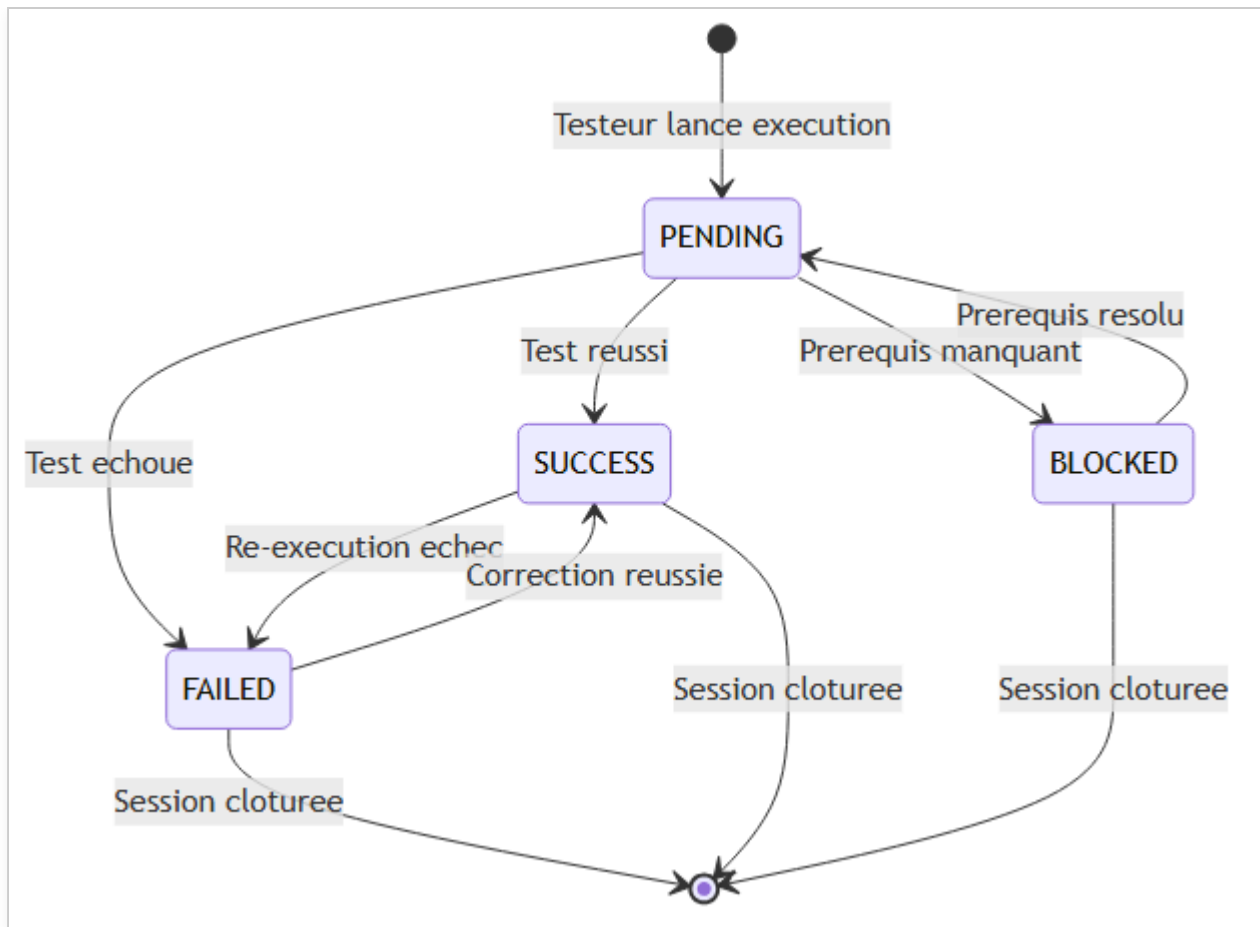


Figure 22 — Diagramme d'etats-transitions : Execution

7. Diagramme de composants

Le diagramme de composants modelise l'architecture logicielle du systeme en identifiant les principaux composants et leurs dependances. Il offre une vue de haut niveau de l'organisation du code.

7.1. Couche Frontend (React - Port 5173)

Le frontend est organise en trois groupes de composants :

- **Composants IA :**
 - *GenerateurCasTestModal* : modal permettant de saisir une description et de generer des cas de test via l'IA.
 - *GraviteBadgeIA* : composant affichant la suggestion de gravite avec l'indice de confiance.
 - *SessionResumePanel* : panneau affichant le resume intelligent genere par l'IA lors de la cloture de session.
- **Pages :** Pages Admin, Pages Testeur et Pages Manager, chacune adaptee au role de l'utilisateur connecte.
- **Services API Axios :** couche de communication HTTP gerant les appels REST vers le backend avec injection automatique du token JWT.

7.2. Couche Backend (Spring Boot - Port 8081)

Le backend suit une architecture en couches classique :

- **Spring Security JWT :** filtre de securite interceptant chaque requete pour valider le token JWT et verifier les autorisations.
- **Controleurs REST :** AuthController, ProjetController, ExecutionController, AnomalieController et AiController, chacun exposant les endpoints de son domaine fonctionnel.
- **Services :** AuthService, ProjetService, ExecutionService pour la logique metier classique, et AiCasTestService, SessionSummaryService, AnomalieGraviteService et OllamaChatbotService pour les fonctionnalites IA.

7.3. Services externes

- **MySQL 8 (testdb)** : base de donnees relationnelle stockant toutes les entites du systeme.
- **Cloudinary CDN** : service cloud pour l'hebergement des captures d'ecran jointes aux executions.
- **Ollama LLM (Port 11434)** : serveur d'inference IA local executant le modele deepseek-r1:7b.

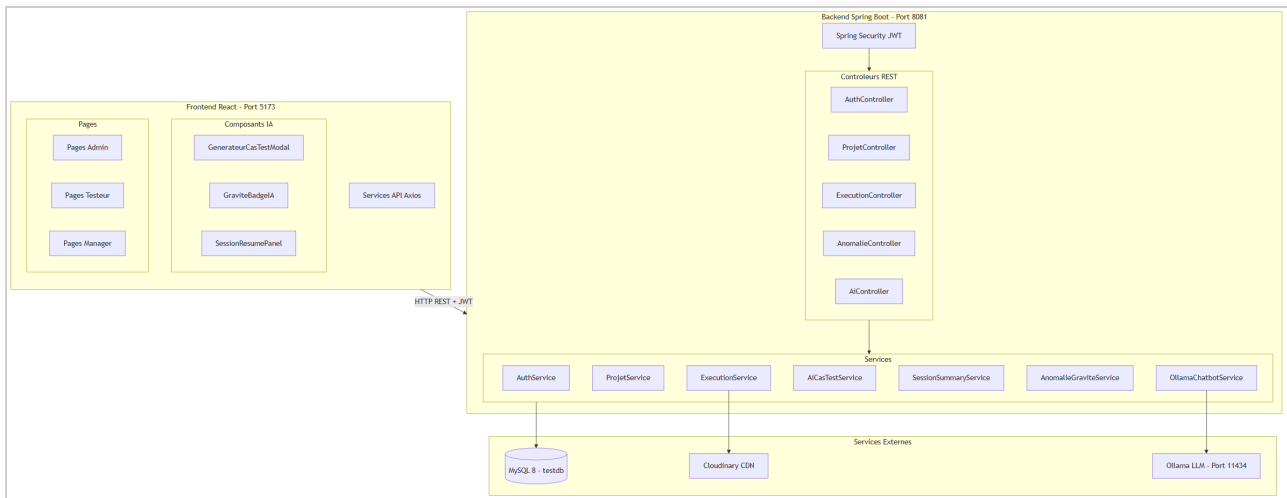


Figure 23 — Diagramme de composants

8. Diagramme de deployment

Le diagramme de deployment modelise l'infrastructure physique du systeme et la repartition des composants logiciels sur les noeuds materiels. Il decrit les protocoles de communication utilises.

8.1. Noeud Client

Le navigateur web (Chrome ou Firefox) constitue le point d'acces de l'utilisateur. Il communique avec le serveur via HTTP sur le port 5173.

8.2. Machine Serveur

La machine serveur heberge quatre composants :

- **Frontend (Port 5173)** : application React 18 servie par Vite, utilisant Tailwind CSS pour le style et Recharts pour les graphiques du dashboard.
- **Backend (Port 8081)** : application Spring Boot 3.2.5 executee sur Java 17. Communique avec le frontend via REST + JWT.

- **Base de donnees (Port 3306) :** serveur MySQL 8.0 hebergeant la base "testdb". Le backend y accede via JDBC.
- **IA Locale (Port 11434) :** serveur Ollama executant les modeles deepseek-r1:7b. Le backend communique avec Ollama via HTTP REST.

8.3. Services Cloud

Cloudinary CDN est le seul service heberge dans le cloud. Le backend y accede via HTTPS pour l'upload et la recuperation des images.

8.4. Protocoles de communication

Source	Destination	Protocole	Port
Navigateur	Frontend React	HTTP	5173
Frontend React	Backend Spring Boot	HTTP REST + JWT	8081
Backend	MySQL	JDBC	3306
Backend	Ollama	HTTP REST	11434
Backend	Cloudinary	HTTPS	443

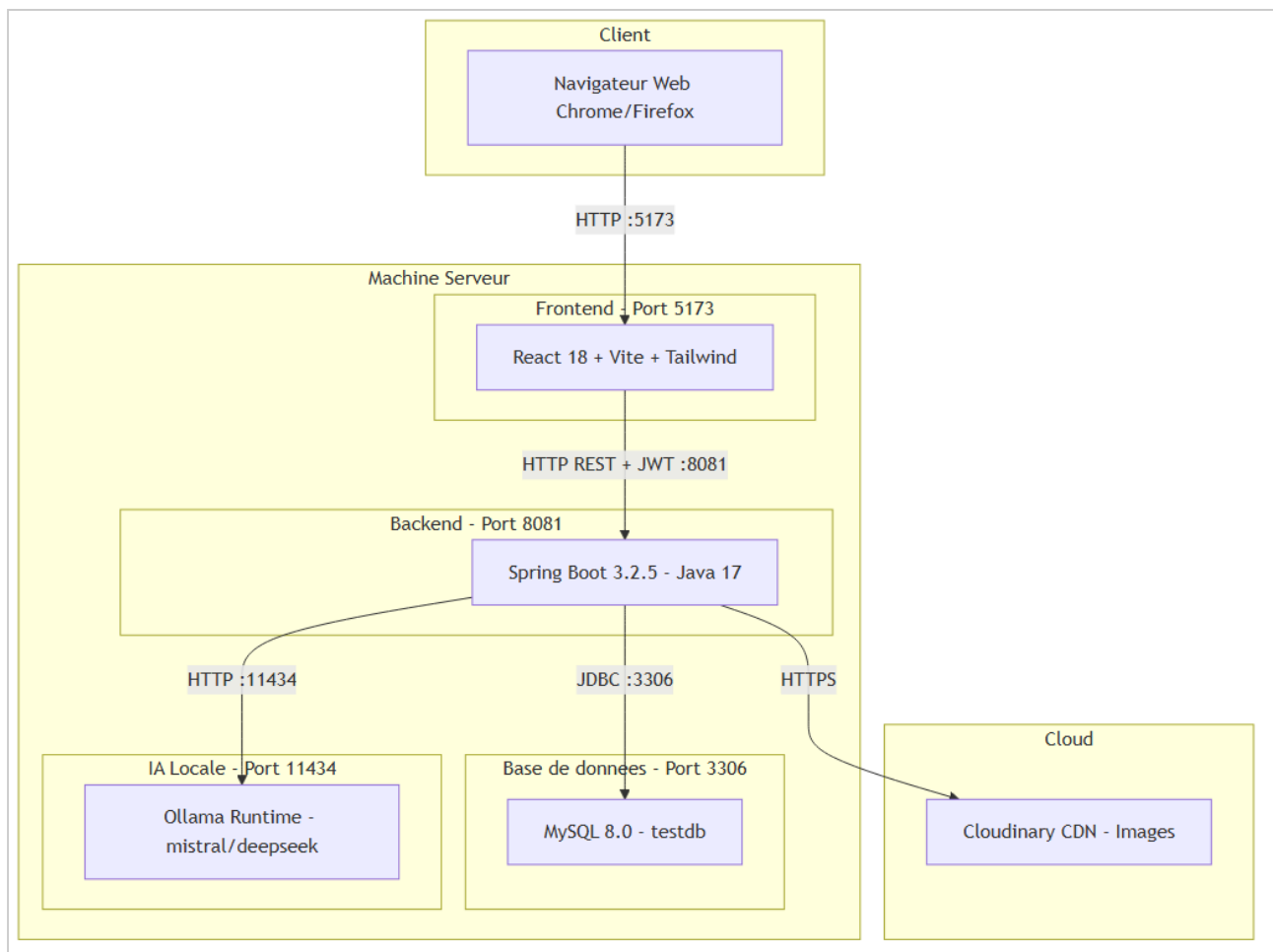


Figure 24 — Diagramme de déploiement

9. Synthèse

Ce chapitre a présente la conception détaillée de la plateforme Test Management Platform a travers 24 diagrammes UML couvrant l'ensemble des perspectives du système.

Type de diagramme	Nombre	Objectif
Cas d'utilisation	12	Decrire les fonctionnalites et les droits d'accès par rôle
Classes	1	Modeliser la structure statique et les relations entre entites
Sequence	5	Illustrer les interactions dynamiques pour les scenarios clés
Activite	1	Représenter le flux de travail global du processus de test
Etats-transitions	3	Modeliser les cycles de vie des entites Projet, Anomalie et Execution
Composants	1	Decrire l'architecture logicielle et les dependances
Deploiement	1	Représenter l'infrastructure physique et les protocoles
Total	24	

La conception met en evidence plusieurs points forts de l'architecture :

- **Separation stricte des responsabilites** : chaque rôle (Admin, Manager, Testeur) dispose de permissions clairement definies et verifiees par Spring Security via les annotations `@PreAuthorize`.
- **Architecture en couches** : la separation entre controleurs, services et repositories favorise la maintenabilite et la testabilite du code.
- **Integration IA innovante** : les trois fonctionnalites d'intelligence artificielle (generation de cas de test, resume de session, suggestion de gravite) apportent une valeur ajoutée significative en automatisant des taches repetitives et en assistant la prise de decision.
- **Scalabilite** : l'architecture modulaire permet d'ajouter facilement de nouveaux modules ou services sans impacter l'existant.

Cette conception detaillee sert de fondation solide pour l'implementation presentee dans le chapitre suivant.